

Primer on Optimization and Optimal Control

Marco Frego, Free University of Bozen-Bolzano

1 From Control to Optimal Control

2 Derivatives

- Analytic Derivatives
- Derivatives via Finite Differences (FD)
- Algorithmic Differentiation - AD
- Reverse Algorithmic Differentiation
- Another interpretation

3 Integration

- Gaussian quadrature
- Radau and Lobatto rules
- From quadrature to ODE integrators
- Runge–Kutta methods
- Stability and practical behaviour

4 NonLinear Programming

5 Solvers and algorithms

6 Direct Methods

- Special Cases

1. From Control to Optimal Control

The **dynamics** of a system is represented usually by an ODE:

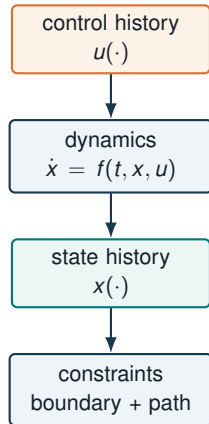
$$\dot{x}(t) = f(t, x(t), u(t)), \quad x(0) = x_0,$$

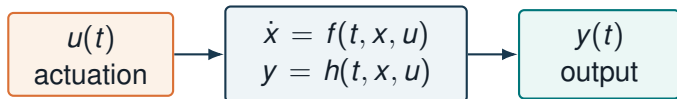
with endpoint and path requirements such as

$$b(x(0), x(T), T) = 0, \quad c(t, x(t), u(t)) \leq 0.$$

- Choose an **admissible** function $u(\cdot)$.
- The dynamics generate the corresponding $x(\cdot)$.
- Feasibility means that all boundary and path constraints hold.

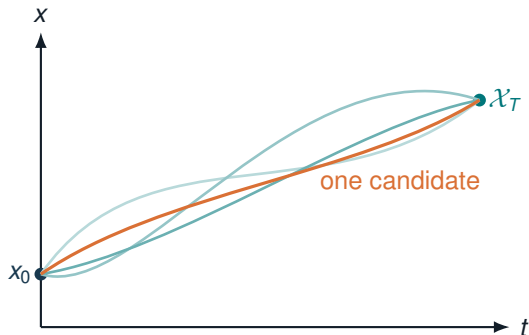
In Control we seek a feasible u , typically, there are **infinitely many** of such u .





- The **state** is sufficient to predict future motion once future controls are specified.
- The **control** is the decision function.
- The **output** need not equal the full state.
- Optimal control is naturally posed in state space.

Feasible trajectories form a family



A feasible manoeuvre is a pair $(x(\cdot), u(\cdot))$ satisfying:

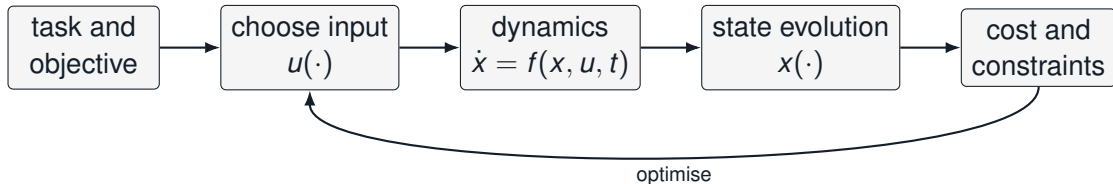
- dynamics;
- admissible controls;
- endpoint conditions;
- path constraints.

The feasible set is typically infinite-dimensional.

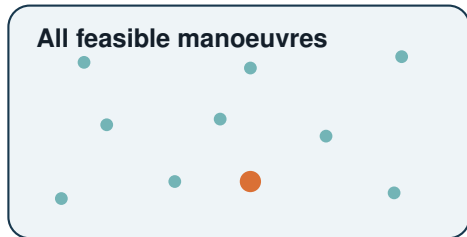
Optimal control in one sentence

Optimal control is the mathematical study of how to choose the inputs of a dynamical system so that the resulting motion is best according to a specified criterion and admissible according to specified constraints.

- The unknown is not only a real number.
- It is usually a function of time: $u(\cdot)$.
- The state $x(\cdot)$ is generated by the dynamics.
- Dynamic models predict how the future state depends on the current state.
- “Best” means lowest cost or highest reward.
- “Admissible” means physically, technologically and legally allowed.
- The formulation is part of the science.



A gallery of objective functionals



minimise J



- Time

$$J_{\text{time}} = T$$

- Tracking

$$J_{\text{track}} = \int_0^T \|x - x_r\|_Q^2 + \|u - u_r\|_R^2 dt$$

- Energy

$$J_{\text{energy}} = \int_0^T u^2(t) dt$$

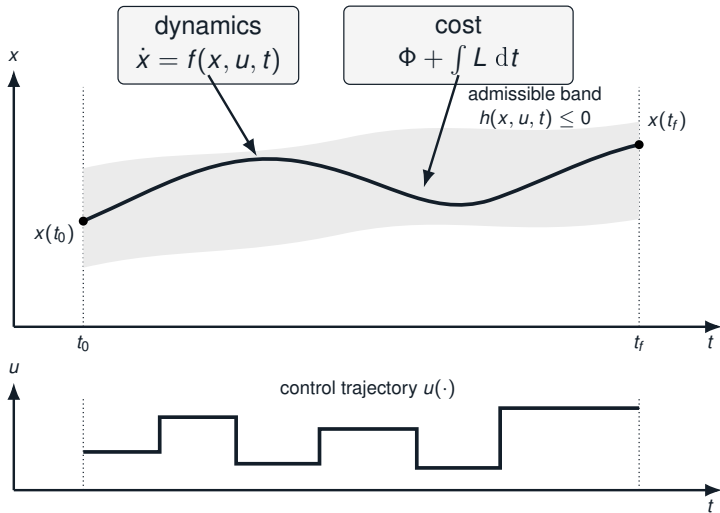
- Risk or economics. Expected loss, fuel, monetary cost, emissions, or a **weighted multi-objective compromise**.

The standard objective to be minimised, in **Bolza form**, is

$$J[x, u, p, t_0, t_f] = \Phi(x(t_0), t_0, x(t_f), t_f, p) + \int_{t_0}^{t_f} L(x(t), u(t), t, p) dt.$$

- $[t_0, t_f]$ is the time interval, typically is $[0, T]$ (with T possibly free).
- Φ is the **endpoint or terminal cost**.
- L is the **running cost**.
- p is a vector of constant parameters that define the problem.
- Multi-objective design is usually scaled by weights or constraints.
- Poor scaling of the cost terms can damage numerical performance.
- The objective function is a modelling choice: it determines what “best” means.

Anatomy of a continuous-time OCP



The objective translates the word “best” into a scalar quantity.

$$J_M[x, u] = \Phi(x(t_0), t_0, x(t_f), t_f)$$

Mayer term,

$$J_L[x, u] = \int_{t_0}^{t_f} L(x(t), u(t), t) dt$$

Lagrange term,

$$J_B[x, u] = J_M[x, u] + J_L[x, u]$$

Bolza form.

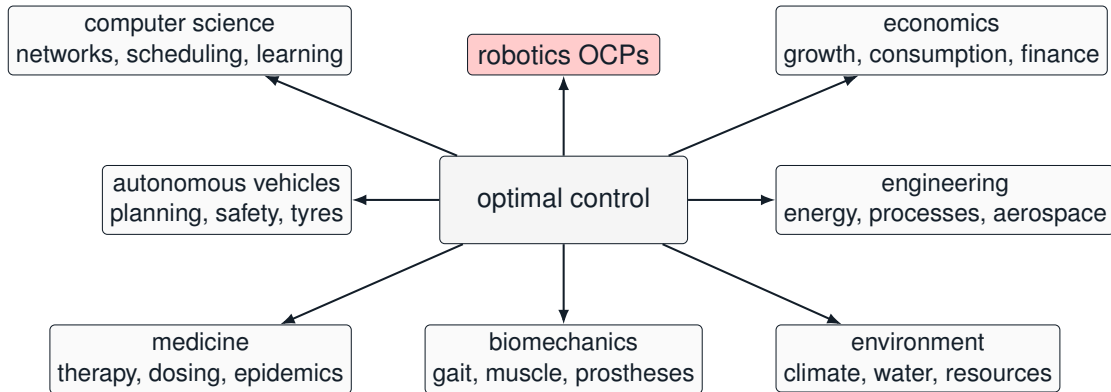
- Minimum-time problems often use $L \equiv 1$ and free t_f .
- Energy-aware problems may penalise $\|u(t)\|^2$ or mechanical work.
- Tracking problems penalise deviations from a reference trajectory.

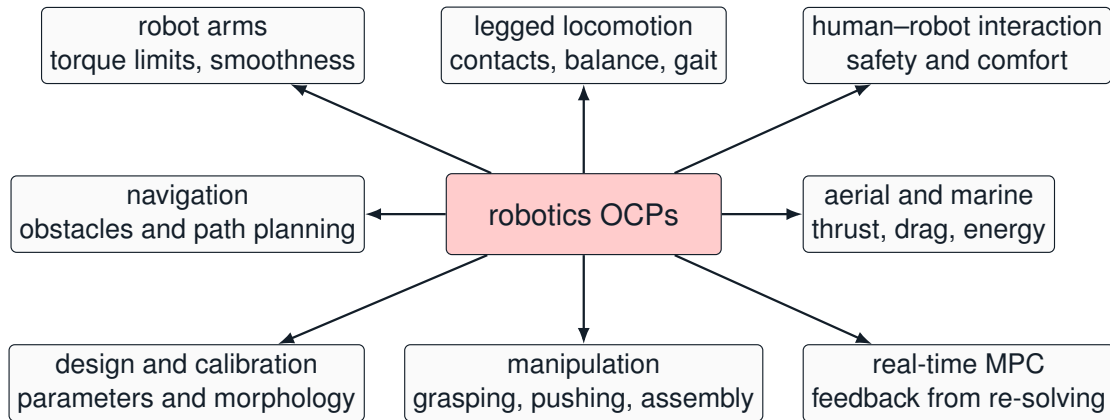
Robots are physical, computational and decision-making systems at the same time. If a physical or safety requirement is missing from the formulation, the optimiser may exploit the omission:

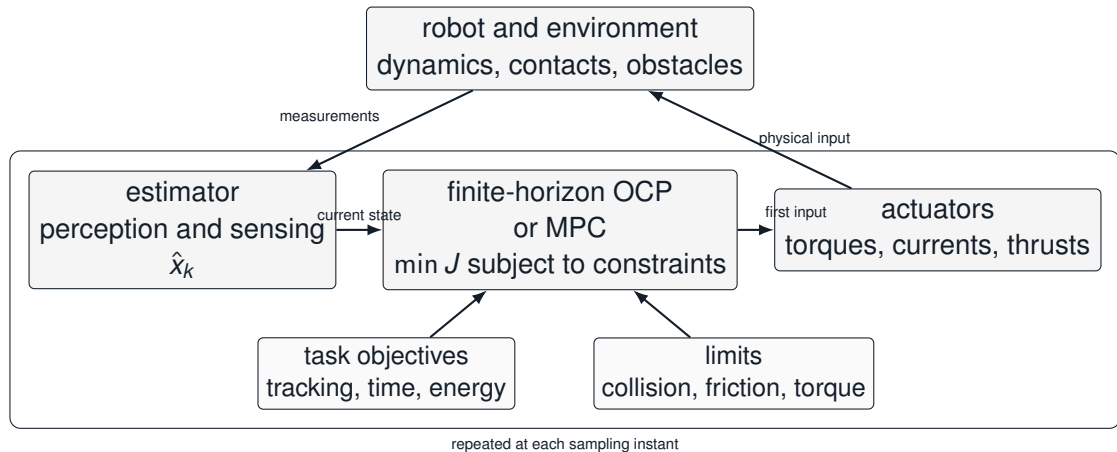
- **Control constraints:** actuator torque, voltage, thrust, current, rate limits.
- **State constraints:** joint limits, temperatures, battery charge, obstacle clearance.
- **Path constraints:** friction cones, centre-of-mass support, safety distance, comfort.
- **Boundary constraints:** start and target poses, periodic gait, terminal sets.
- **Mixed constraints:** relations involving both state and control, e.g. tyre-force limits.

For robotics, an OCP is not only an abstract optimisation problem; it is a way of specifying what a robot should do and what it must never do.

Application domains at a glance







A general OCP may optimise over

$$x(\cdot), \quad u(\cdot), \quad p, \quad t_0, \quad t_f.$$

- In fixed-time problems, t_0 and t_f are prescribed.
- In minimum-time problems, t_f is usually a decision variable.
- The control may be constrained pointwise by $u(t) \in \mathcal{U}(t)$.
- The state may be constrained pointwise by $x(t) \in \mathcal{X}(t)$.
- Hybrid OCPs add discrete modes, switches or contact schedules.

For a first introduction, the continuous-state, continuous-control case is the cleanest starting point.

The state must satisfy

$$\dot{x}(t) = f(x(t), u(t), t, p) \quad \text{for almost every } t \in [t_0, t_f].$$

- For ODEs, standard existence and uniqueness results require regularity in (x, u, t) , for instance local Lipschitz continuity in x .
- Differential-algebraic equations (DAE) add algebraic variables and constraints.
- Partial differential equations (PDE) arise in fluids, heat, waves and distributed systems.
- Stochastic dynamics (SDE) add noise and lead to stochastic optimal control.

The dynamic constraint is what distinguishes an OCP from ordinary finite-dimensional optimisation.

Boundary conditions couple the endpoints and possibly the initial and terminal times:

$$b(x(t_0), t_0, x(t_f), t_f, p) = 0,$$

- Initial value: $x(t_0) = x_0$.
- Fixed target: $x(t_f) = x_f$.
- Terminal set: $x(t_f) \in \mathcal{X}_f$.
- Periodic condition: $x(t_f) = x(t_0)$.
- Some endpoint components may be free (e.g., unknown minimum time t_f), leading to additional conditions in indirect theory (transversality).

Path equality constraints have the form

$$g(x(t), u(t), t, p) = 0 \quad \text{for almost every } t \in [t_0, t_f].$$

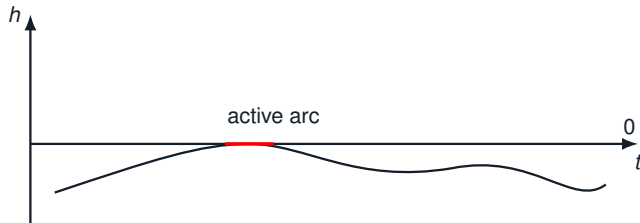
- They may represent holonomic constraints, closed kinematic loops or algebraic model equations.
- In robotics they can encode contact sticking, rolling without slipping or task-space equalities.
- They are often delicate numerically because they reduce the feasible manifold.
- In direct methods they become algebraic equality constraints at nodes, intervals or collocation points.

The phrase “for almost every time” allows isolated switching or corner points in many formulations.

Path inequality constraints have the form

$$h(x(t), u(t), t, p) \leq 0 \quad \text{for almost every } t \in [t_0, t_f].$$

- Examples: obstacle clearance, actuator saturation and safety distances.
- A constraint is *active* when equality holds at the optimum.
- Active constraints introduce multipliers and changes the structure of optimal arcs.
- Checking constraints only at mesh points can miss between-node violations.



Simple bounds (box constraints) are important enough to write explicitly:

$$u_{\min}(t) \leq u(t) \leq u_{\max}(t), \quad x_{\min}(t) \leq x(t) \leq x_{\max}(t).$$

- Actuator saturation often produces bang-bang or boundary-arc solutions.
- State bounds encode safe operating envelopes and hardware limits.
- They improve physical realism but can make the numerical problem harder.
- Bounds on torque, velocity, joint-position and collision-avoidance are essential.

Constraints are not an afterthought; they define the feasible world in which optimality is meaningful.

$$\begin{aligned} \min_{x(\cdot), u(\cdot), p, t_0, t_f} \quad & \Phi(x(t_0), t_0, x(t_f), t_f, p) + \int_{t_0}^{t_f} L(x(t), u(t), t, p) \, dt \\ \text{s.t.} \quad & \dot{x}(t) = f(x(t), u(t), t, p) && \text{a.e. } t \in [t_0, t_f], \\ & g(x(t), u(t), t, p) = 0 && \text{a.e. } t \in [t_0, t_f], \\ & h(x(t), u(t), t, p) \leq 0 && \text{a.e. } t \in [t_0, t_f], \\ & b(x(t_0), t_0, x(t_f), t_f, p) = 0, \\ & x(t) \in \mathcal{X}(t), \quad u(t) \in \mathcal{U}(t). \end{aligned}$$

This template may be specialised to fixed or free times, multiple phases, hybrid systems, stochastic dynamics, DAEs or PDEs.

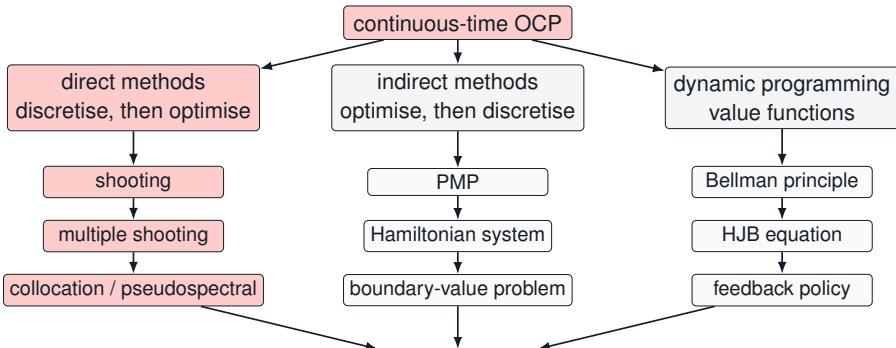
We will see the fixed time, single phase, deterministic, ODE case.

Not every written OCP has a meaningful or attainable solution.

- The feasible set can be empty.
- The cost can be unbounded below or the infimum may not be attained.
- Discontinuous controls, state constraints and hybrid switches create issues.
- Convexity, coercivity, continuity and compactness are typical existence ingredients.
- A feasible process satisfies all dynamics, boundary conditions and constraints.
- A globally optimal process has the smallest cost over the entire feasible set.
- A locally optimal process has the smallest cost in a neighbourhood of the process.
- Most non-linear numerical methods compute local solutions unless special global methods are used.

A good formulation is a mathematical model, not merely a list of equations.

Classical routes to solve an OCP

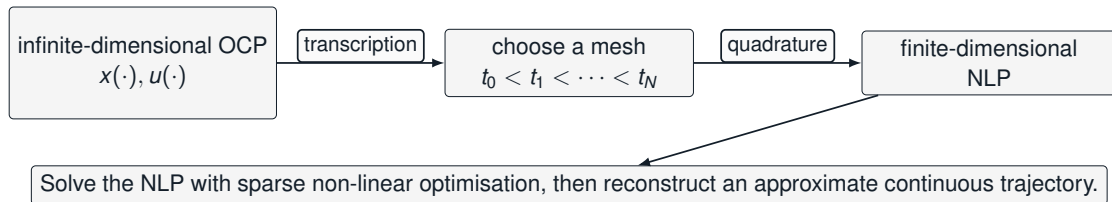


Hybrid practice: MPC, adjoints inside direct solvers, DDP/iLQR, convex relaxations, robust and stochastic OCPs.

In this course we will see the Direct Method approach.

Direct transcription: the basic idea

Within the philosophy “first discretise, then optimise”:



To tackle with the Direct Method an OCP, we need to understand some concepts:

- How to compute **derivatives** $\implies$ Analytic derivatives, Finite Differences (FD) and Automatic Differentiation (AD).
- How to compute **integrals** $\implies$ integration/quadrature methods (Euler, Runge Kutta, Spectral methods).
- How to compute the solution of an **ODE** (shooting, collocation).
- How to solve the resulting **NonLinear Programming** (NLP).

These are the topics of the next sections.

2. Derivatives

2. Derivatives

2.1 Analytic Derivatives

Let $F : \mathbb{R}^m \rightarrow \mathbb{R}^p$. For an explicit expression, apply the chain, product, and quotient rules exactly. The Jacobian J_F is a $p \times m$ matrix:

$$F(u_1, u_2) = \begin{bmatrix} \sin(u_1 u_2) \\ u_1^2 + \exp(u_2) \end{bmatrix} \implies J_F(u) = \begin{bmatrix} u_2 \cos(u_1 u_2) & u_1 \cos(u_1 u_2) \\ 2u_1 & \exp(u_2) \end{bmatrix}.$$

Computer algebra systems (CAS):

Maple, Mathematica, SymPy, and symbolic graphs can generate closed-form derivative code.

Cautions:

Expression swell, branch simplification, removable singularities, and target-code quality can dominate runtime.

Symbolic differentiation manipulates formulas; automatic differentiation differentiates the executed elementary operations.

2. Derivatives

2.2 Derivatives via Finite Differences (FD)

For a direction v , the **forward finite difference** is

$$J_F(u)v \approx \frac{F(u + \epsilon v) - F(u)}{\epsilon}.$$

For the full $p \times m$ Jacobian, use $v = e_i$ for $i = 1, \dots, m$:

$$J_F(u) \approx \left[\frac{F(u + \epsilon e_1) - F(u)}{\epsilon} \quad \dots \quad \frac{F(u + \epsilon e_m) - F(u)}{\epsilon} \right].$$

- truncation error: $O(\epsilon)$;
- cancellation / round-off error: roughly $O(\varepsilon_{\text{mach}}/\epsilon)$;
- cost: about $m + 1$ evaluations for a dense Jacobian;
- universal advantage: works with a deterministic black box;
- useful if F is an experiment or a measurement (e.g., velocity from GPS data).

Central difference

$$J_F(u)v \approx \frac{F(u + \epsilon v) - F(u - \epsilon v)}{2\epsilon},$$

with $O(\epsilon^2)$ truncation error but two function calls per direction.

Complex step

For analytic code that accepts complex arithmetic,

$$J_F(u)v \approx \frac{\Im F(u + i\epsilon v)}{\epsilon}.$$

It avoids subtractive cancellation and permits tiny ϵ .

Complex step fails through nonanalytic operations such as $|\cdot|$, comparisons, clipping, or code that discards imaginary parts.

Example - forward differences

We keep the two-variable example introduced before and fix $\epsilon = 10^{-8}$:

$$F(u_1, u_2) = \begin{bmatrix} \sin(u_1 u_2) \\ u_1^2 + \exp(u_2) \end{bmatrix}, \quad J_F(u) = \begin{bmatrix} u_2 \cos(u_1 u_2) & u_1 \cos(u_1 u_2) \\ 2u_1 & \exp(u_2) \end{bmatrix}, \quad u^* = \begin{bmatrix} \pi/4 \\ 1 \end{bmatrix}.$$

For the coordinate direction e_j , the **forward** finite-difference column is

$$J_{:j} = \frac{F(u^* + \epsilon e_j) - F(u^*)}{\epsilon}.$$

Nominal value

$$F(u^*) = F\left(\frac{\pi}{4}, 1\right) = \begin{bmatrix} 0.7071067812 \\ 3.3351321035 \end{bmatrix}.$$

Perturbation samples

$$F(u^* + \epsilon e_1) = \begin{bmatrix} 0.7071067883 \\ 3.3351321192 \end{bmatrix},$$

$$F(u^* + \epsilon e_2) = \begin{bmatrix} 0.7071067867 \\ 3.3351321307 \end{bmatrix}.$$

Example - numerical Jacobian

Using $\varepsilon = 10^{-8}$, the two columns are obtained as

$$J_{:,1} \approx \frac{F(u^* + \varepsilon e_1) - F(u^*)}{\varepsilon} = \begin{bmatrix} 0.7071067842 \\ 1.5707963819 \end{bmatrix}, \quad J_{:,2} \approx \frac{F(u^* + \varepsilon e_2) - F(u^*)}{\varepsilon} = \begin{bmatrix} 0.5553603688 \\ 2.7182818663 \end{bmatrix}.$$

Forward finite-difference result

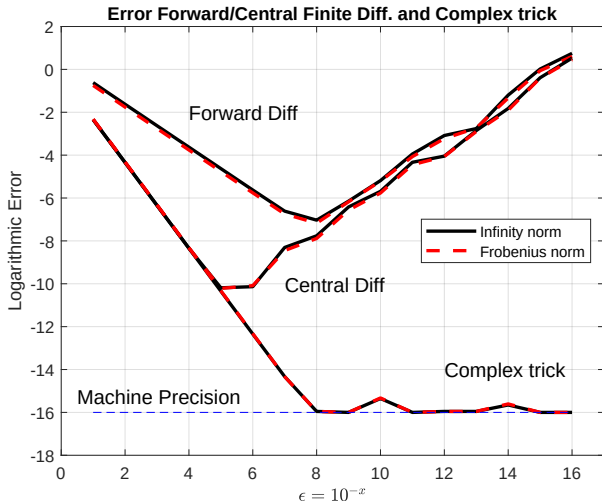
$$J(u^*) \approx \begin{bmatrix} 0.7071067842 & 0.5553603688 \\ 1.5707963819 & 2.7182818663 \end{bmatrix}.$$

Exact Jacobian

$$J_F(u^*) = \begin{bmatrix} 0.7071067812 & 0.5553603673 \\ 1.5707963268 & 2.7182818285 \end{bmatrix}.$$

entry	J_{11}	J_{12}	J_{21}	J_{22}
absolute error	$3.05 \cdot 10^{-9}$	$1.53 \cdot 10^{-9}$	$5.51 \cdot 10^{-8}$	$3.78 \cdot 10^{-8}$

Forward FD are black-box and simple: one nominal call plus one perturbed call per input direction. The price is first-order truncation error and sensitivity to round-off when ε is very small.



The minimum occurs near

$$\epsilon \sim \sqrt{\epsilon_{\text{mach}}}$$

for a first-order forward difference after scaling. Typically, $\epsilon = 10^{-8}$.

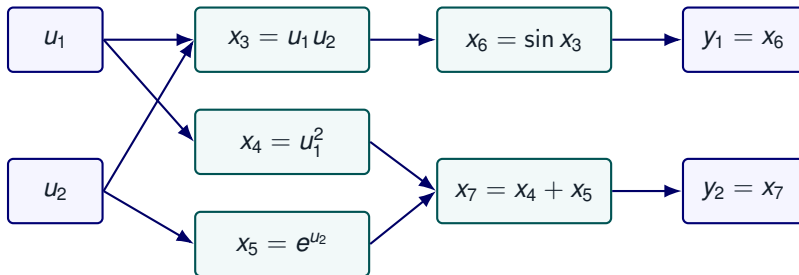
No single absolute ϵ works for variables with wildly different units and magnitudes.

2. Derivatives

2.3 Algorithmic Differentiation - AD

Algorithmic (automatic) Differentiation (AD) is a different method to compute derivatives. If the function is represented by a computer code, AD differentiates each step of it. It starts assuming the function F is written as a sequence of elementary operations. Let

$$F(u_1, u_2) = \begin{bmatrix} \sin(u_1 u_2) \\ u_1^2 + \exp(u_2) \end{bmatrix} \Rightarrow \begin{bmatrix} u_1, u_2 & \rightarrow & \text{input var} \\ x_1, \dots, x_7 & \rightarrow & \text{intermediate var} \\ y_1, y_2 & \rightarrow & \text{output var} \end{bmatrix}$$



The complete forward sweep

	Primal evaluation	Forward derivative
1	$x_1 = u_1$	$\dot{x}_1 = \dot{u}_1$
2	$x_2 = u_2$	$\dot{x}_2 = \dot{u}_2$
3	$x_3 = x_1 x_2$	$\dot{x}_3 = x_2 \dot{x}_1 + x_1 \dot{x}_2$
4	$x_4 = x_1^2$	$\dot{x}_4 = 2x_1 \dot{x}_1$
5	$x_5 = \exp x_2$	$\dot{x}_5 = \exp(x_2) \dot{x}_2$
6	$x_6 = \sin x_3$	$\dot{x}_6 = \cos(x_3) \dot{x}_3$
7	$x_7 = x_4 + x_5$	$\dot{x}_7 = \dot{x}_4 + \dot{x}_5$
	$y_1 = x_6, y_2 = x_7$	$\dot{y}_1 = \dot{x}_6, \dot{y}_2 = \dot{x}_7$

Each tangent line is at most a small constant factor longer than its primal companion. Intermediate pairs $(x_i, \dot{x}_i)$ may be discarded after last use. Exact up to machine precision.

Consider the usual function F of the previous examples at $u^* = (\pi/4, 1)^T$. We write its evaluation as a sequence of elementary operations for $v = \dot{u} = \mathbf{e}_1$, then $J_{:,1}$:

	primal variable	tangent variable	x_i	$\dot{x}_i$
1	$x_1 = u_1$	$\dot{x}_1 = \dot{u}_1$	0.7853981634	1
2	$x_2 = u_2$	$\dot{x}_2 = \dot{u}_2$	1	0
3	$x_3 = x_1 x_2$	$\dot{x}_3 = x_2 \dot{x}_1 + x_1 \dot{x}_2$	0.7853981634	1
4	$x_4 = x_1^2$	$\dot{x}_4 = 2x_1 \dot{x}_1$	0.6168502751	1.5707963268
5	$x_5 = \exp x_2$	$\dot{x}_5 = \exp(x_2) \dot{x}_2$	2.7182818285	0
6	$x_6 = \sin x_3$	$\dot{x}_6 = \cos(x_3) \dot{x}_3$	0.7071067812	0.7071067812
7	$x_7 = x_4 + x_5$	$\dot{x}_7 = \dot{x}_4 + \dot{x}_5$	3.3351321035	1.5707963268
$y_1 = x_6, \quad y_2 = x_7 \quad \dot{y}_1 = \dot{x}_6, \quad \dot{y}_2 = \dot{x}_7$			$\begin{bmatrix} 0.7071067812 \\ 3.3351321035 \end{bmatrix}$	$\begin{bmatrix} 0.7071067812 \\ 1.5707963268 \end{bmatrix}$

Forward AD - 2nd column of J_F

The second column of J_F at $u^* = (\pi/4, 1)^T$ is obtained for $v = \dot{u} = e_2$, then $J_{:,2}$:

	primal variable	tangent variable	x_i	$\dot{x}_i$
1	$x_1 = u_1$	$\dot{x}_1 = \dot{u}_1$	0.7853981634	0
2	$x_2 = u_2$	$\dot{x}_2 = \dot{u}_2$	1	1
3	$x_3 = x_1 x_2$	$\dot{x}_3 = x_2 \dot{x}_1 + x_1 \dot{x}_2$	0.7853981634	0.7853981634
4	$x_4 = x_1^2$	$\dot{x}_4 = 2x_1 \dot{x}_1$	0.6168502751	0
5	$x_5 = \exp x_2$	$\dot{x}_5 = \exp(x_2) \dot{x}_2$	2.7182818285	2.7182818285
6	$x_6 = \sin x_3$	$\dot{x}_6 = \cos(x_3) \dot{x}_3$	0.7071067812	0.5553603673
7	$x_7 = x_4 + x_5$	$\dot{x}_7 = \dot{x}_4 + \dot{x}_5$	3.3351321035	2.7182818285
$y_1 = x_6, \quad y_2 = x_7 \quad \dot{y}_1 = \dot{x}_6, \quad \dot{y}_2 = \dot{x}_7$			$\begin{bmatrix} 0.7071067812 \\ 3.3351321035 \end{bmatrix}$	$\begin{bmatrix} 0.5553603673 \\ 2.7182818285 \end{bmatrix}$

If one line of the primal code is

$$x_k = \varphi_k(x_i, x_j),$$

then its forward-mode companion is

$$\dot{x}_k = \frac{\partial \varphi_k}{\partial x_i} \dot{x}_i + \frac{\partial \varphi_k}{\partial x_j} \dot{x}_j.$$

Primal operation	Tangent operation	Local rule
$z = a + b$	$\dot{z} = \dot{a} + \dot{b}$	linearity
$z = ab$	$\dot{z} = b\dot{a} + a\dot{b}$	product
$z = \sin a$	$\dot{z} = \cos(a)\dot{a}$	chain
$z = \exp a$	$\dot{z} = \exp(a)\dot{a}$	chain
$z = a/b$	$\dot{z} = (b\dot{a} - a\dot{b})/b^2$	quotient

Primal value and its derivative can be evaluated side by side; no global expression required.

Collect the straight-line equations in $G(x, u) = 0$ and write the outputs as $y = Cx$.

$$G(x, u) = \begin{bmatrix} x_1 - u_1 \\ x_2 - u_2 \\ x_3 - x_1 x_2 \\ x_4 - x_1^2 \\ x_5 - \exp(x_2) \\ x_6 - \sin(x_3) \\ x_7 - x_4 - x_5 \end{bmatrix} = 0.$$

The output (x_6, x_7) is selected by

$$y = Cx, \quad C = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Hence

$$F(u) = Cx(u)$$

and, in particular, $J(u) = C\dot{x}(u)$.

Differentiating $G(x(u), u) = 0$ with respect to u , along the direction $\dot{u} = v$:

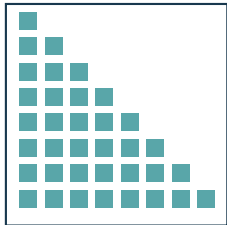
$$G_x \dot{x} + G_u \dot{u} = 0 \implies G_x \dot{x} = Bv, \quad B := -G_u.$$

$$G_x = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ -x_2 & -x_1 & 1 & 0 & 0 & 0 & 0 \\ -2x_1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & -\exp(x_2) & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -\cos(x_3) & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & -1 & 0 & 1 \end{bmatrix} \quad B := \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

Indeed, $\dot{x} = G_x^{-1} Bv$ and multiplying by matrix C yields $CG_x^{-1} B = C\dot{x} = J$.

A triangular solve

Operation k depends only on earlier variables, hence G_x is lower triangular:



G_x : lower triangular

1. compute $\dot{x}_1, \dot{x}_2, \dots$ in order;
2. choose a direction $\dot{u} = v$;
3. never form G_x^{-1} , because G_x is lower triangular;
4. read out $\dot{y} = C\dot{x}$.

Interpretation: the forward AD sweep is the structured linear solve associated with the computational graph. Since G_x is lower triangular, $\dot{x}$ is computed in the same order as the original program.

Generic forward-sweep algorithm

input: $u \in \mathbb{R}^m$, seed $v \in \mathbb{R}^m$

for $i = 1, \dots, m$:

$x_i \leftarrow u_i$

$\dot{x}_i \leftarrow v_i$

for $k = m + 1, \dots, n$:

$x_k \leftarrow \varphi_k(x_{j_1}, x_{j_2})$

$\dot{x}_k \leftarrow \varphi_{k,x_{j_1}} \dot{x}_{j_1} + \varphi_{k,x_{j_2}} \dot{x}_{j_2}$

$y \leftarrow Cx$; $\dot{y} \leftarrow C\dot{x}$

Key properties

- one pass in primal order;
- local derivatives only;
- the code can stream values and tangents together;
- output is one Jv , not necessarily the full J ;
- requires m (independent) sweeps to build the full Jacobian.

Forward mode is especially natural for state sensitivities and maps with few input directions.

One direction

The conservative bound is

$$\text{cost}(J_F v) \lesssim 2 \text{cost}(F).$$

The practical constant is often smaller.

Full dense Jacobian

With m unit seeds,

$$\frac{\text{cost}(F, J_F)}{\text{cost}(F)} \lesssim 1 + 2m.$$

Forward mode scales with input directions.

Memory and error

Values and tangents can be streamed together. Memory can be about twice the primal evaluation, with no step-size or truncation error.

choose by derivative-product shape



2. Derivatives

2.4 Reverse Algorithmic Differentiation

The Reverse or Adjoint mode of AD moves backward in the graph: we apply the chain rule through the computational graph, but the derivative information flows backward, from output to input. The forward sweep is still required to build the graph.

The forward derivative satisfies the equation obtained differentiating $G(x, u)$:

$$G_x \dot{x} = Bv,$$

the reverse mode solves the transposed system:

$$G_x^T \bar{x} = C^T \bar{y}, \quad \bar{u} = B^T \bar{x}$$

where the bar-variables are different from the dot-variables of forward mode. To compute $\bar{x}$ we need to solve an upper-triangular system by a backward sweep.

Reverse mode computes a vector-Jacobian product $\bar{u} = J_F(u)^T \bar{y}$. For

$$F(u_1, u_2) = \begin{bmatrix} \sin(u_1 u_2) \\ u_1^2 + \exp(u_2) \end{bmatrix},$$

we first run the primal sweep (as in forward mode) and store

$$x_1 = u_1, \quad x_2 = u_2, \quad x_3 = x_1 x_2, \quad x_4 = x_1^2, \quad x_5 = e^{x_2}, \quad x_6 = \sin x_3, \quad x_7 = x_4 + x_5.$$

Reverse sweep. Start from an output seed $\bar{y} = \begin{bmatrix} \bar{y}_1 \\ \bar{y}_2 \end{bmatrix}$.

Since $y_1 = x_6$ and $y_2 = x_7$, initialise

$$\bar{x}_6 = \bar{y}_1, \quad \bar{x}_7 = \bar{y}_2.$$

All other adjoints are initially zero.

Process the elementary operations backwards (+ = means update current value):

$$x_7 = x_4 + x_5 \implies \bar{x}_4 += \bar{x}_7, \quad \bar{x}_5 += \bar{x}_7.$$

$$x_6 = \sin x_3 \implies \bar{x}_3 += \cos(x_3) \bar{x}_6.$$

$$x_5 = \exp(x_2) \implies \bar{x}_2 += \exp(x_2) \bar{x}_5.$$

$$x_4 = x_1^2 \implies \bar{x}_1 += 2x_1 \bar{x}_4.$$

$$x_3 = x_1 x_2 \implies \bar{x}_1 += x_2 \bar{x}_3, \quad \bar{x}_2 += x_1 \bar{x}_3.$$

Finally,

$$\bar{u}_1 = \bar{x}_1, \quad \bar{u}_2 = \bar{x}_2.$$

Collecting the contributions gives

$$\bar{u}_1 = x_2 \cos(x_3) \bar{y}_1 + 2x_1 \bar{y}_2$$

$$\bar{u}_2 = x_1 \cos(x_3) \bar{y}_1 + \exp(x_2) \bar{y}_2.$$

Since

$$x_1 = u_1, \quad x_2 = u_2, \quad x_3 = u_1 u_2,$$

we obtain

$$\begin{bmatrix} \bar{u}_1 \\ \bar{u}_2 \end{bmatrix} = \begin{bmatrix} u_2 \cos(u_1 u_2) & 2u_1 \\ u_1 \cos(u_1 u_2) & \exp(u_2) \end{bmatrix} \begin{bmatrix} \bar{y}_1 \\ \bar{y}_2 \end{bmatrix}$$

that is,

$$\bar{u} = J_F(u)^\top \bar{y}.$$

$$F(u_1, u_2) = \begin{bmatrix} \sin(u_1 u_2) \\ u_1^2 + \exp(u_2) \end{bmatrix}, \quad u^* = \begin{bmatrix} \pi/4 \\ 1 \end{bmatrix}.$$

$$x_1 = \pi/4 = 0.7853981634,$$

$$x_2 = 1,$$

$$x_3 = x_1 x_2 = \pi/4 = 0.7853981634,$$

$$x_4 = x_1^2 = \pi^2/16 = 0.6168502751,$$

$$x_5 = \exp(x_2) = e = 2.7182818285,$$

$$x_6 = \sin(x_3) = \sqrt{2}/2 = 0.7071067812,$$

$$x_7 = x_4 + x_5 = 3.3351321035.$$

Thus

$$F(u^*) = \begin{bmatrix} x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 0.7071067812 \\ 3.3351321035 \end{bmatrix}.$$

Reverse mode computes

$$\bar{u} = J_F(u^*)^\top \bar{y}.$$

Seed $\bar{y} = e_1$. This computes the gradient of y_1 :

$$\bar{x}_6 = 1, \quad \bar{x}_7 = 0.$$

Then

$$\bar{x}_3 = \cos(x_3)\bar{x}_6 = \frac{\sqrt{2}}{2}, \quad \bar{x}_4 = 0, \quad \bar{x}_5 = 0.$$

Finally

$$\bar{u}_1 = x_2\bar{x}_3 + 2x_1\bar{x}_4 = \frac{\sqrt{2}}{2}, \quad \bar{u}_2 = x_1\bar{x}_3 + \exp(x_2)\bar{x}_5 = \frac{\pi\sqrt{2}}{8}.$$

$$J_F(u^*)^\top e_1 = \begin{bmatrix} 0.7071067812 \\ 0.5553603673 \end{bmatrix}.$$

Seed $\bar{y} = e_2$. This computes the gradient of y_2 :

$$\bar{x}_6 = 0, \quad \bar{x}_7 = 1.$$

Thus

$$\bar{x}_4 = 1, \quad \bar{x}_5 = 1, \quad \bar{x}_3 = 0.$$

Finally

$$\bar{u}_1 = 2x_1 = \frac{\pi}{2}, \quad \bar{u}_2 = \exp(x_2) = e.$$

$$J_F(u^*)^\top e_2 = \begin{bmatrix} 1.5707963268 \\ 2.7182818285 \end{bmatrix}.$$

One reverse seed

The cost of reverse mode is bound by

$$\text{cost}(J_F^T w) \leq 3 \text{cost}(F).$$

Full Jacobian

Using the p output unit seeds,

$$\text{cost}(F, J_F) \leq (1 + 3p) \text{cost}(F).$$

The bound is independent of m !

Memory price

The primal intermediates and local derivative data must be stored or reconstructed for the reverse pass.

Why optimisation and machine learning use it

For a scalar objective ($p = 1$), one primal evaluation plus one reverse sweep gives the entire gradient. This result is known as the *cheap-gradient result* and links it to backpropagation.

2. Derivatives

2.5 Another interpretation

Let $F : \mathbb{R}^m \rightarrow \mathbb{R}^p$ be evaluated as a composition:

$$F = f_N \circ f_{N-1} \circ \cdots \circ f_1.$$

Introduce intermediate variables

$$x_0 = u, \quad x_1 = f_1(x_0), \quad x_2 = f_2(x_1), \quad \dots, \quad x_N = f_N(x_{N-1}) = y.$$

The chain rule gives

$$J_F(u) = J_{f_N}(x_{N-1}) J_{f_{N-1}}(x_{N-2}) \cdots J_{f_1}(x_0).$$

Main idea

Forward and reverse AD differ only in how this product of Jacobians is applied.

Choose an input direction

$$v = \dot{u}.$$

Forward AD computes

$$\dot{y} = J_F(u)v.$$

Using the chain rule:

$$\dot{x}_0 = v, \quad \dot{x}_1 = J_{f_1}(x_0)\dot{x}_0, \quad \dot{x}_2 = J_{f_2}(x_1)\dot{x}_1,$$

and so on until

$$\dot{y} = J_{f_N}(x_{N-1})\dot{x}_{N-1}.$$

Thus

$$v \mapsto J_{f_1} v \mapsto J_{f_2} J_{f_1} v \mapsto \cdots \mapsto J_F v.$$

Geometric interpretation: Forward AD is the push-forward of a tangent vector.

Choose an output seed

$$w = \bar{y} \in \mathbb{R}^p.$$

Reverse AD computes

$$\bar{u} = J_F(u)^\top w.$$

Since

$$J_F(u)^\top = J_{f_1}(x_0)^\top J_{f_2}(x_1)^\top \cdots J_{f_N}(x_{N-1})^\top,$$

we propagate backwards:

$$\bar{x}_N = w,$$

$$\bar{x}_{N-1} = J_{f_N}(x_{N-1})^\top \bar{x}_N,$$

$$\bar{x}_{N-2} = J_{f_{N-1}}(x_{N-2})^\top \bar{x}_{N-1},$$

and continue until

$$\bar{u} = J_{f_1}(x_0)^\top \bar{x}_1.$$

Forward AD : input perturbation $v \mapsto$ output perturbation $J_F(u)v$,

Reverse AD : output sensitivity $w \mapsto$ input sensitivity $J_F(u)^\top w$.

Forward mode is efficient when we need a few columns of the Jacobian.

$$v = e_j \implies J_F(u)e_j = J_{:,j}(u).$$

Reverse mode is efficient when we need gradients of scalar outputs. If $F : \mathbb{R}^m \rightarrow \mathbb{R}$, then choosing $w = 1$ gives $J_F(u)^\top w = \nabla F(u)$.

Optimisation viewpoint: For a scalar objective, one reverse sweep propagates the objective sensitivity back to all input variables.

3. Integration

Part I: numerical quadrature

Replace a continuous integral by a weighted sum of function values:

$$\int_a^b f(x) dx \approx \sum_{i=1}^m w_i f(x_i).$$

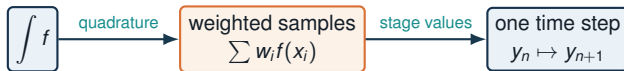
Accuracy depends on the nodes, the weights, regularity and interval subdivision.

Part II: ODE integrators

Write one ODE step as an integral:

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} F(t, y(t)) dt.$$

Different quadrature ideas generate Euler, trapezoidal, collocation and Runge–Kutta methods.



Quadr.: area by weighted samples

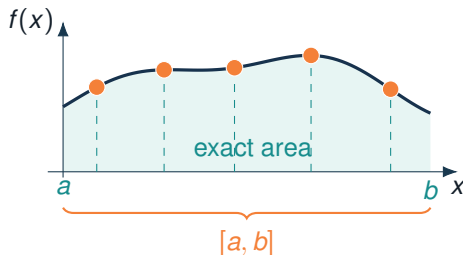
For a sufficiently regular function,

$$I(f) = \int_a^b f(x) dx.$$

A quadrature rule uses only finitely many data:

$$Q_m(f) = \sum_{i=1}^m w_i f(x_i),$$

$$x_i \in [a, b], \quad i = 1, \dots, m.$$



The error functional is

$$E_m(f) = I(f) - Q_m(f).$$

Nodes determine *where* the function is sampled; weights determine *how much* each sample contributes.

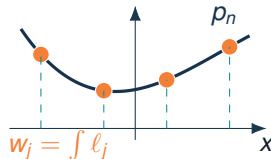
Select distinct nodes $x_0, \dots, x_n$ and interpolate

$$p_n(x) = \sum_{j=0}^n f(x_j) \ell_j(x), \quad \ell_j(x) = \prod_{k \neq j} \frac{x - x_k}{x_j - x_k}.$$

Integrating p_n gives

$$Q_{n+1}(f) = \sum_{j=0}^n w_j f(x_j), \quad w_j = \int_a^b \ell_j(x) dx.$$

- the rule is automatically exact on ∂n polynomials;
- special symmetry can increase the degree;
- interpolation error is integrated, not just evaluated.



Integration nodes can be prescribed or selected by the method

Newton–Cotes

Interpolate at **equally spaced nodes**.

$$x_j = a + j \cdot \frac{b - a}{n} = a + jh.$$

Low-order members include rectangle, midpoint, trapezoidal. Composite rules apply them repeatedly on short panels.

Regular spacing useful but loses degrees of accuracy.



Gaussian quadrature

The nodes are determined as part of the numerical method.

Their position is uneven but they ensure the maximum degree of accuracy for the same number of function evaluations.

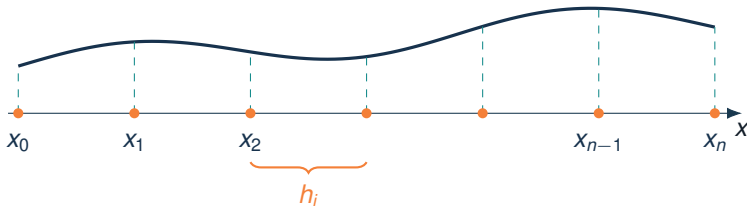


Partition

$$a = x_0 < x_1 < \cdots < x_n = b, \quad h_i = x_{i+1} - x_i.$$

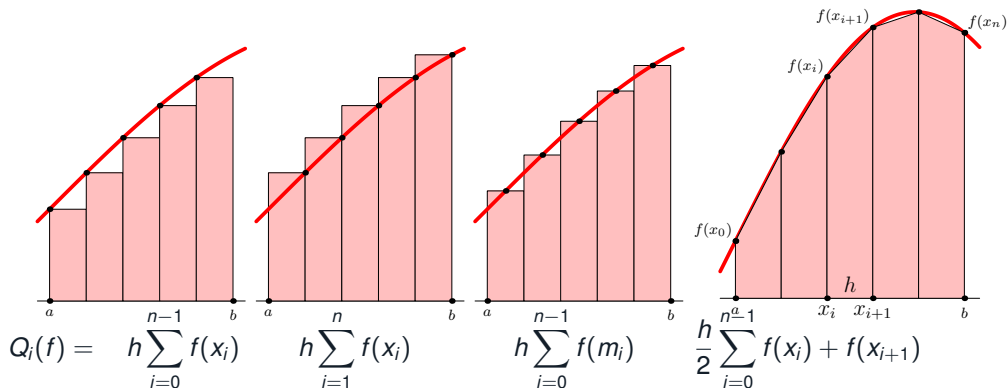
Apply a low-order formula on each subinterval and add:

$$\int_a^b f(x) \, dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) \, dx \approx \sum_{i=0}^{n-1} Q_i(f).$$



Classic Newton-Cotes formulas

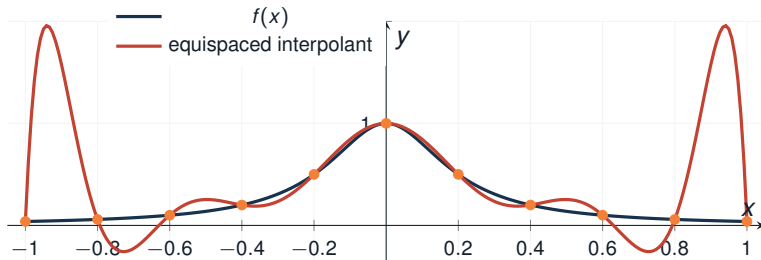
For (usually) equally spaced nodes, the left/right rectangle rule, the midpoint rule and the trapezoid method are very popular. Let $h = \frac{b-a}{n} = x_{i+1} - x_i$ and $m_i = \frac{x_i + x_{i+1}}{2}$.



A classic counterexample is to interpolate the Runge function

$$f(x) = \frac{1}{1 + 25x^2}, \quad -1 \leq x \leq 1,$$

at 11 equally spaced nodes. The degree-10 interpolant matches every sample but oscillates strongly near ± 1 .



3. Integration

3.1 Gaussian quadrature

With m nodes, a general rule contains $2m$ parameters (the x_i and the w_i):

$$Q_m(f) = \sum_{i=1}^m w_i f(x_i).$$

Gaussian quadrature selects both $\{x_i\}$ and $\{w_i\}$ so that

$$Q_m(p) = \int_{-1}^1 p(x) dx \quad \forall p \in \mathcal{P}_{2m-1} \text{ polynomials.}$$

Maximum possible degree

No m -node rule can be exact for every polynomial of degree $2m$: the polynomial $\prod_{i=1}^m (x - x_i)^2$ vanishes at every node but has a strictly positive integral.



Legendre polynomials satisfy

$$\int_{-1}^1 P_m(x) P_k(x) dx = 0 \quad (m \neq k), \quad P_m(1) = 1.$$

For the roots x_i of P_m ,

$$w_i = \frac{2}{(1 - x_i^2) [P'_m(x_i)]^2}.$$

If $f \in C^{2m}[-1, 1]$, the error has the form

$$I(f) - Q_m(f) = \frac{2^{2m+1} (m!)^4}{(2m+1) [(2m)!]^3} f^{(2m)}(\xi).$$

Consequence

Smoothness is converted efficiently into accuracy: m samples integrate every polynomial through degree $2m - 1$.

The first Gauss–Legendre rules

m	Nodes x_i	Weights w_i	Degree
1	0	2	1
2	$\pm 1/\sqrt{3}$	1, 1	3
3	$0, \pm\sqrt{3/5}$	8/9, 5/9, 5/9	5

For example, the two-point rule gives

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$



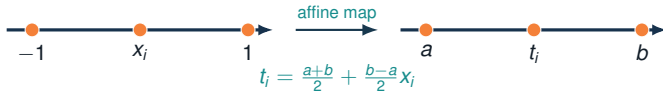
Affine change of variable

Map $x \in [-1, 1]$ to $t \in [a, b]$ by

$$t = \frac{a+b}{2} + \frac{b-a}{2}x, \quad dt = \frac{b-a}{2}dx.$$

Therefore

$$\int_a^b f(t) dt \approx \frac{b-a}{2} \cdot \sum_{i=1}^m w_i f\left(\frac{a+b}{2} + \frac{b-a}{2}x_i\right).$$



Example: 2 Gauss points on $[0, 1]$

Apply the two-point rule to

$$I = \int_0^1 e^t dt = e - 1 \approx 1.718281828459.$$

The transformed nodes are

$$t_{1,2} = \frac{1}{2} \left(1 \mp \frac{1}{\sqrt{3}} \right), \quad \tilde{w}_{1,2} = \frac{1}{2}.$$

Hence

$$Q_2 = \frac{1}{2} \left[\exp\left(\frac{1}{2} - \frac{1}{2\sqrt{3}}\right) + \exp\left(\frac{1}{2} + \frac{1}{2\sqrt{3}}\right) \right] \approx 1.717896378008,$$
$$|I - Q_2| \approx 3.85 \cdot 10^{-4}.$$

With three Gauss points, $Q_3 \approx 1.718281004373$, $|I - Q_3| \approx 8.24 \cdot 10^{-7}$.

3. Integration

3.2 Radau and Lobatto rules

An m -node Gauss–Radau rule includes one prescribed endpoint and has degree

$$2m - 2.$$

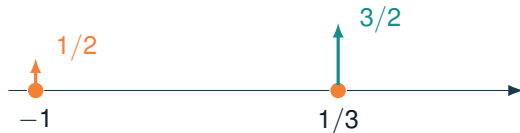
For the left endpoint $x = -1$, the nodes are the roots of

$$P_m(x) + P_{m-1}(x),$$

including the root -1 . For the right endpoint $x = 1$, use $P_m(x) - P_{m-1}(x)$.

The two-node left Radau rule is

$$\int_{-1}^1 f(x) dx \approx \frac{1}{2}f(-1) + \frac{3}{2}f\left(\frac{1}{3}\right).$$



It is exact through degree 2.

Gauss–Lobatto fixes both endpoints

An m -node Gauss–Lobatto rule includes $x_1 = -1$ and $x_m = 1$. Its $m - 2$ interior nodes are the roots of P'_{m-1} and its degree is

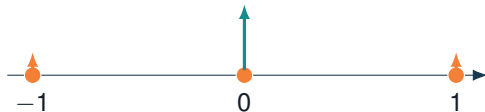
$$2m - 3.$$

The weights are

$$w_1 = w_m = \frac{2}{m(m-1)}, \quad w_i = \frac{2}{m(m-1)[P'_{m-1}(x_i)]^2}.$$

For $m = 3$,

$$x = (-1, 0, 1), \quad w = \left(\frac{1}{3}, \frac{4}{3}, \frac{1}{3}\right).$$



Endpoints: accuracy for structure

Family	m nodes	Endpoints	Degree	Typical reason to choose it
Gauss–Legendre	m	none	$2m - 1$	maximum degree for a fixed number of samples
Gauss–Radau	m	one	$2m - 2$	include a known initial or final value
Gauss–Lobatto	m	both	$2m - 3$	continuity across panels; both boundary values available

The best rule is not determined by degree alone: endpoint access, nesting, smoothness and reuse of samples can dominate.

3. Integration

3.3 From quadrature to ODE integrators

An ODE step is an integral

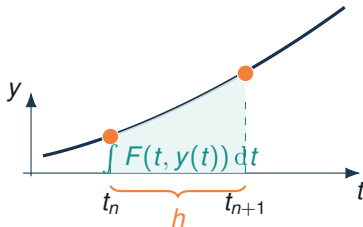
Consider the initial-value problem

$$y'(t) = F(t, y(t)), \quad y(t_0) = y_0.$$

Integrating from t_n to $t_{n+1} = t_n + h$ gives the exact relation

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} F(t, y(t)) dt.$$

A numerical method replaces the integral and, when necessary, the unknown solution values inside the integrand.



Approximate the integral by its left endpoint:

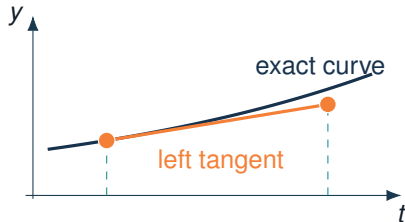
$$\int_{t_n}^{t_{n+1}} F(t, y(t)) dt \approx hF(t_n, y_n).$$

Therefore

$$y_{n+1} = y_n + hF(t_n, y_n).$$

- explicit: no equation is solved;
- one function evaluation per step;
- local truncation error $\mathcal{O}(h^2)$;
- global order 1.

The numerical solution follows the tangent at the beginning of each step.



Use the integrand at the unknown right endpoint:

$$\int_{t_n}^{t_{n+1}} F(t, y(t)) \, dt \approx hF(t_{n+1}, y_{n+1}).$$

Then

$$y_{n+1} = y_n + hF(t_{n+1}, y_{n+1}).$$

- implicit: solve a generally nonlinear equation;
- global order 1;
- excellent damping for stiff dissipative problems;
- Jacobians and Newton iterations may be required.

Define the residual

$$G(z) = z - y_n - hF(t_{n+1}, z).$$

Newton's method computes

$$[I - hF_y(t_{n+1}, z^{(k)})]\Delta z = -G(z^{(k)}),$$

$$z^{(k+1)} = z^{(k)} + \Delta z.$$

3. Integration

3.4 Runge–Kutta methods

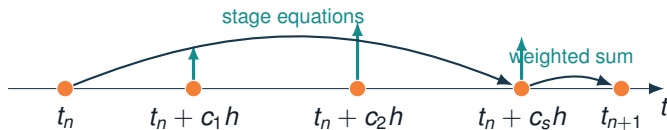
Runge–Kutta use internal stages

An s-stage Runge–Kutta method computes

$$Y_i = y_n + h \sum_{j=1}^s a_{ij} F(t_n + c_j h, Y_j), \quad i = 1, \dots, s,$$

and updates

$$y_{n+1} = y_n + h \sum_{j=1}^s b_j F(t_n + c_j h, Y_j).$$



- b_j are quadrature-like weights;
- c_j locate the stages in time;
- a_{ij} determine how stage states approximate the unknown trajectory.

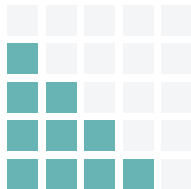
$$\begin{array}{c|c} c & A \\ \hline & b^T \end{array} \quad c_i = \sum_{j=1}^s a_{ij} \quad \text{for a consistent method.}$$

Explicit method

A is strictly lower triangular:

$$a_{ij} = 0 \quad j \geq i.$$

Stages are computed sequentially without nonlinear solves.



strictly lower-triangular A

Implicit method

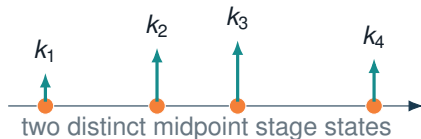
A has diagonal or upper-triangular entries. Stages are coupled and require linear or nonlinear solves, but stability can improve dramatically.

Classical RK4 yields 4th-order acc.

$$\begin{aligned}k_1 &= F(t_n, y_n), \\k_3 &= F(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2), \\y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).\end{aligned}$$

$$\begin{aligned}k_2 &= F(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1), \\k_4 &= F(t_n + h, y_n + hk_3),\end{aligned}$$

0	0	0	0	0
1/2	1/2	0	0	0
1/2	0	1/2	0	0
1	0	0	1	0
	1/6	1/3	1/3	1/6



Choose nodes $0 \leq c_1, \dots, c_s \leq 1$ and seek a polynomial $u(t)$ satisfying

$$u(t_n) = y_n, \quad u'(t_n + c_i h) = F(t_n + c_i h, u(t_n + c_i h)).$$

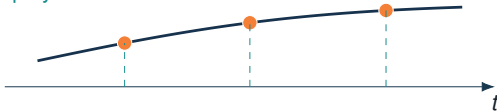
If ℓ_j are the cardinal polynomials for the c_j , define

$$a_{ij} = \int_0^{c_i} \ell_j(\tau) d\tau, \quad b_j = \int_0^1 \ell_j(\tau) d\tau.$$

Then the collocation equations are exactly

$$Y_i = y_n + h \sum_{j=1}^s a_{ij} F(t_n + c_j h, Y_j), \quad y_{n+1} = y_n + h \sum_{j=1}^s b_j F_j.$$

the polynomial satisfies the ODE at the collocation nodes



Choose the s Gauss–Legendre nodes mapped to $(0, 1)$. The resulting Runge–Kutta method has order $2s$.

- A-stable;
- symmetric;
- symplectic;
- neither endpoint is a collocation node;
- effective for smooth non-stiff or Hamiltonian dynamics.

$s = 3$: order 6



One stage

$c_1 = 1/2$ gives the implicit midpoint rule.

Two-stage Gauss meth. is 4th order

The mapped Gauss nodes are

$$c_{1,2} = \frac{1}{2} \mp \frac{\sqrt{3}}{6}.$$

Integrating their cardinal polynomials gives

$\frac{1}{2} - \frac{\sqrt{3}}{6}$	$\frac{1}{4}$	$\frac{1}{4} - \frac{\sqrt{3}}{6}$
$\frac{1}{2} + \frac{\sqrt{3}}{6}$	$\frac{1}{4} + \frac{\sqrt{3}}{6}$	$\frac{1}{4}$
	$\frac{1}{2}$	$\frac{1}{2}$

Two coupled stage equations must be solved at every step, but the method achieves order 4 with only two stage evaluations.

Geometric property

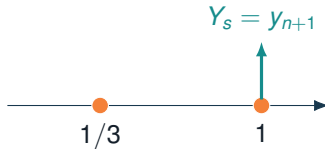
Applied to Hamiltonian systems, Gauss methods preserve the symplectic structure and usually show excellent long-time energy behaviour.

Radau IIA strongly damps stiffness

Radau IIA collocation includes $c_s = 1$ and has order $2s - 1$. For $s = 2$,

$1/3$	$5/12$	$-1/12$
1	$3/4$	$1/4$
	$3/4$	$1/4$

- A-stable and L-stable;
- stiffly accurate: $y_{n+1} = Y_s$;
- excellent for stiff differential equations and DAEs.



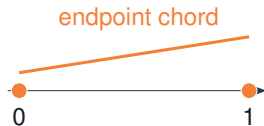
Lobatto IIIA includes both ends

Lobatto collocation uses $c_1 = 0$ and $c_s = 1$ and has order $2s - 2$. For $s = 2$,

0	0	0
1	1/2	1/2
	1/2	1/2

which is exactly the trapezoidal method.

- both endpoint slopes are represented;
- adjacent steps share endpoint information;
- related Lobatto pairs are useful in partitioned and constrained dynamics.



Quadrature types

Collocation family	Nodes in $[0, 1]$	Order	Stability	Characteristic use
Gauss	interior	$2s$	A-stable	high order, symmetry, symplectic integration
Radau IIA	includes 1	$2s - 1$	L-stable	stiff ODEs and DAEs; strong damping
Lobatto IIIA	includes 0, 1	$2s - 2$	A-stable	endpoint coupling and partitioned methods

Gauss

Radau IIA

Lobatto IIIA

3. Integration

3.5 Stability and practical behaviour

Take

$$y' = -y^2, \quad y(0) = 1, \quad h = \frac{1}{2}.$$

Explicit Euler gives

$$y_1^{\text{EE}} = 1 - \frac{1}{2} = \frac{1}{2}.$$

Implicit Euler requires

$$y_1 = 1 - \frac{1}{2}y_1^2 \iff y_1^2 + 2y_1 - 2 = 0.$$

Selecting the positive root,

$$y_1^{\text{IE}} = \sqrt{3} - 1 \approx 0.732051.$$

The exact value is $y(1/2) = 2/3 \approx 0.666667$.

Explicit Euler underestimates the positive solution.

Implicit Euler overestimates it but preserves positivity for any $h > 0$.

Compute two approximations of different accuracy from related data:

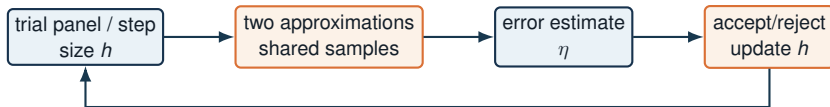
$$Q_{\text{fine}}, Q_{\text{coarse}} \quad \text{or} \quad y_{n+1}^{[\rho]}, y_{n+1}^{[\rho-1]}.$$

Their difference estimates the local error:

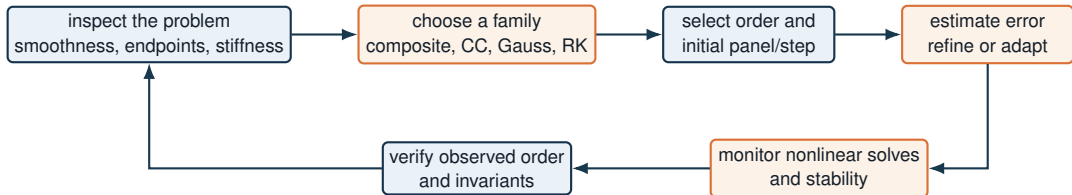
$$\eta_Q = |Q_{\text{fine}} - Q_{\text{coarse}}|, \quad \eta_{\text{RK}} = \|y_{n+1}^{[\rho]} - y_{n+1}^{[\rho-1]}\|.$$

If the estimator behaves as $\eta = \mathcal{O}(h^q)$, a standard update is

$$h_{\text{new}} = \text{safety} \cdot h \left(\frac{\text{tol}}{\eta} \right)^{1/q}.$$

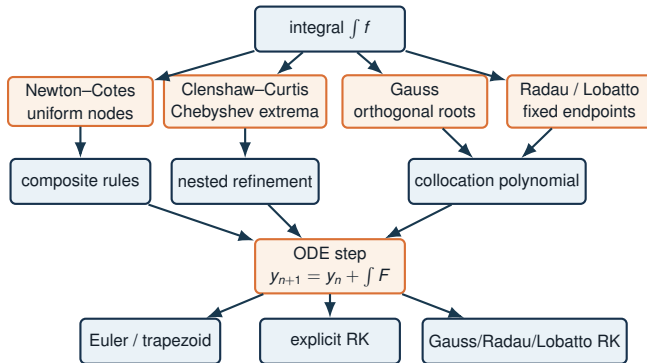


Workflow: regularity, acc., stability



- quadrature: split at nonsmooth points and compare successive refinements;
- ODEs: distinguish accuracy-limited from stability-limited step sizes;
- implicit methods: include algebraic-solve tolerances in the error budget;
- always test the code on problems with known integrals or exact solutions.

The main connections

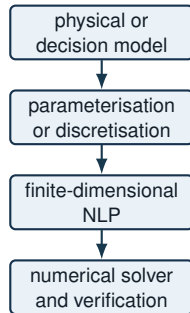


4. NonLinear Programming

Many models begin in a specialised language:

- optimal control and trajectory planning;
- estimation, design and allocation;
- equilibrium, contact and hybrid systems.

After parameterisation or discretisation, they often become **finite-dimensional NLPs**.



Let $x \in \mathbb{R}^n$ denote the vector of decision variables. The standard smooth NLP is

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \quad h : \mathbb{R}^n \rightarrow \mathbb{R}^{m_e}, \\ & g(x) \leq 0, \quad g : \mathbb{R}^n \rightarrow \mathbb{R}^{m_i}, \\ & x_L \leq x \leq x_U. \end{aligned}$$

Objective

$f(x)$ measures cost, time, energy, error, risk, or a weighted compromise.

Equalities

$h(x) = 0$ impose exact relations: dynamics, balances, geometry, conservation.

Inequalities

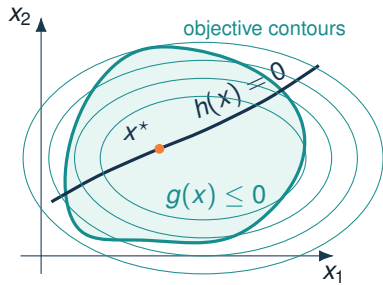
$g(x) \leq 0$ encode limits, safety margins, capacities and admissible regions.

The feasible set is

$$\mathcal{F} = \{x \in \mathbb{R}^n : h(x) = 0, g(x) \leq 0, x_L \leq x \leq x_U\}.$$

A point $x^* \in \mathcal{F}$ is optimal if it has the smallest objective value among the admissible competitors under consideration.

- Equalities usually reduce dimension.
- Inequalities create boundaries and corners.



The **Active Set** at a feasible point x is

$$\mathcal{A}(x) = \{i \in \{1, \dots, m\} : g_i(x) = 0\}.$$

- $g_i(x) < 0$: the constraint is **inactive**; locally there is slack.
- $g_i(x) = 0$: the constraint is **active**; its normal can oppose descent.
- Bounds are inequalities too and may enter the active set.

Active-set identification is a central task in constrained optimisation.

Local minimiser

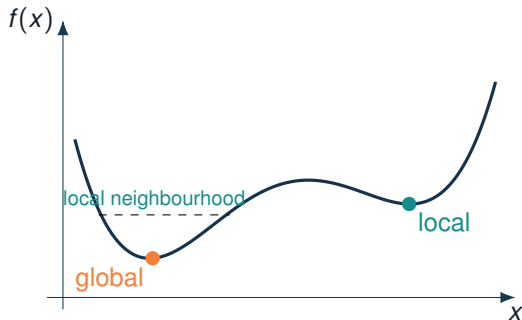
$x^* \in \mathcal{F}$ is locally optimal if there exists $\varepsilon > 0$ such that

$$f(x^*) \leq f(x) \quad \forall x \in \mathcal{F} \cap B_\varepsilon(x^*).$$

Global minimiser

$x^* \in \mathcal{F}$ is globally optimal if

$$f(x^*) \leq f(x) \quad \forall x \in \mathcal{F}.$$



General NLP solvers normally seek a local solution; convexity is what turns local optimality into global optimality.

Around the current point x_k , let $p = x - x_k$. Taylor expansion gives

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T \nabla^2 f(x_k) p,$$

$$h(x_k + p) \approx h(x_k) + J_h(x_k) p,$$

$$g(x_k + p) \approx g(x_k) + J_g(x_k) p.$$

Gradient

Direction of greatest first-order increase of the objective.

Jacobians

Linearise the equality and inequality constraint maps.

Hessian

Describes curvature and distinguishes minima, maxima and saddle points.

Newton, SQP and interior-point methods are built from these local models.

The Lagrangian merges objective and constraints.

For the convention $g(x) \leq 0$, define

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^T h(x) + \mu^T g(x), \quad \mu \geq 0.$$

- $\lambda \in \mathbb{R}^{m_e}$ are equality multipliers.
- $\mu \in \mathbb{R}_+^{m_i}$ are inequality multipliers.
- Multipliers quantify marginal sensitivity to constraint relaxation.

Under a suitable constraint qualification, a local minimiser x^* admits multipliers (λ^*, μ^*) satisfying

$$\begin{aligned}\nabla f(x^*) + J_h(x^*)^T \lambda^* + J_g(x^*)^T \mu^* &= 0 && \text{stationarity,} \\ h(x^*) &= 0, \quad g(x^*) \leq 0 && \text{primal feasibility,} \\ \mu^* &\geq 0 && \text{dual feasibility,} \\ \mu_i^* g_i(x^*) &= 0 \quad \forall i && \text{complementarity.}\end{aligned}$$

KKT is not automatically sufficient

For a general nonconvex NLP, a KKT point may be a local minimiser, a local maximiser, or a saddle point. Curvature or convexity must also be considered.

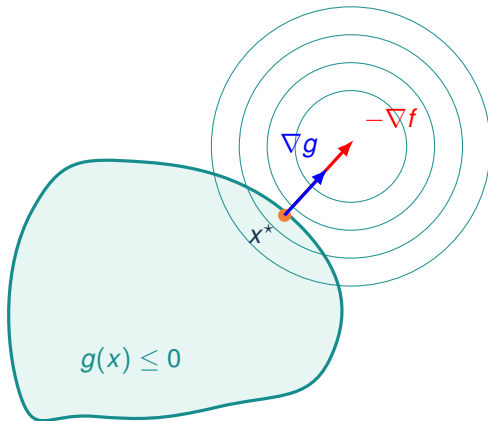
At a constrained optimum

With one active inequality and no equalities,

$$-\nabla f(x^*) = \mu^* \nabla g(x^*), \quad \mu^* \geq 0.$$

The unconstrained descent direction points outside the feasible set; the active constraint supplies the balancing normal. With several active constraints, $-\nabla f$

belongs to the cone generated by their normals, together with the span of equality normals.



A common regularity condition: LICQ

At x^* , the vectors

$$\{\nabla h_j(x^*)\}_{j=1}^{m_e} \cup \{\nabla g_i(x^*)\}_{i \in \mathcal{A}(x^*)}$$

are linearly independent.

LICQ is sufficient for existence and uniqueness of the KKT multipliers locally, but it is stronger than necessary.

Second-order information

Let

$$W^* = \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*, \mu^*).$$

Positive curvature of $d^T W^* d$ along feasible critical directions provides a second-order sufficient condition for a strict local minimum.

Convex case: KKT conditions are sufficient for global optimality under standard regularity assumptions.

Exploit the structure of the problem!

When the problem is not fully a NLP, specific results are derived:

- linear objective and constraints: **Linear Programming** (LP)
- quadratic objective and linear constraint: **Quadratic Programming** (QP)
- quadratic objective and constraint: **Quadratically Constrained QP** (QCQP)
- linear objective and conic constraint: **Second Order Cone Programming** (SOCP)
- semidefinite programming: **SDP**

If, instead, also integer variables are present, the class of problems extends to:

- affine objective and constraints: **Mixed Integer LP** (MILP)
- quadratic objective and affine constraints: **Mixed Integer QP** (MIQP)
- nonlinear objective or constraints: **Mixed Integer NLP** (MINLP)

Exploiting the structure of the problem yields optimised methods and software!

A linear programming has the standard form

$$\min_{x \in \mathbb{R}^n} c^T x \quad \text{s.t.} \quad Ax = b, \quad Gx \leq q.$$

- The feasible set is a polyhedron.
- Every local optimum is global.
- On a nonempty bounded polytope, an optimum is attained at an extreme point.
- Algorithms: simplex and primal–dual interior-point methods.

Typical applications

Network flows, resource allocation, blending, production planning, and linearised models.

Recognisable signature

No products of decision variables; no nonlinear functions of x .

A quadratic programming is

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^T H x + c^T x \quad \text{s.t.} \quad A x = b, \quad G x \leq q, \quad H = H^T.$$

Convex QP

$H \succeq 0$. Every local minimiser is global, and KKT conditions are sufficient.

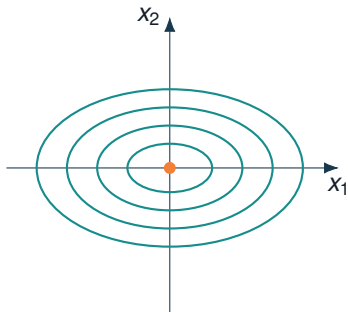
- linear and nonlinear model predictive control;
- constrained least squares;
- portfolio optimisation;
- the local subproblem inside SQP.

Nonconvex QP

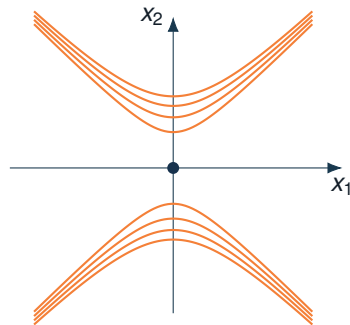
H is indefinite. Multiple local minima and saddle points may occur.

In the usual terminology, a QP already has affine constraints.

The Hessian and QP convexity



$H \succcurlyeq 0$: ellipsoidal contours



H indefinite: hyperbolic contours

Positive semidefiniteness is a structural property, not merely a numerical detail.

Equality-constr. QP $\implies$ linear KKT

Consider

$$\min_x \frac{1}{2}x^T Hx + c^T x \quad \text{s.t.} \quad Ax = b.$$

Its first-order optimality equations are

$$Hx + c + A^T \lambda = 0, \quad Ax - b = 0,$$

hence

$$\begin{bmatrix} H & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}.$$

The matrix is symmetric but generally indefinite. Sparse factorisation and Schur-complement techniques are therefore central.

With inequalities, the active constraints play the role of rows of A ; active-set methods repeatedly solve related KKT systems.

A quadratically constrained quadratic programming has the form

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^T H_0 x + c_0^T x + d_0 \\ \text{s.t.} \quad & \frac{1}{2}x^T H_i x + c_i^T x + d_i \leq 0, \quad i = 1, \dots, m. \end{aligned}$$

If $H_i \succeq 0$ for every inequality and $H_0 \succeq 0$, the problem is convex. Ellipsoidal regions, trust regions and quadratic performance bounds fit naturally.

A second-order cone programming can be written as

$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & \|A_i x + b_i\|_2 \leq d_i^T x + e_i, \quad i = 1, \dots, m, \\ & Fx = q. \end{aligned}$$

- Convex by construction.
- Includes LP as a special case.
- Convex QPs and many convex QCQPs admit SOCP reformulations.

Typical applications

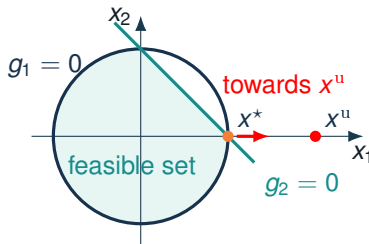
Robust optimisation, friction-cone approximations, portfolio risk, antenna design and norm-bounded estimation.

Consider the two-variable nonlinear programme

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = \frac{1}{2}((x_1 - 2)^2 + x_2^2) \\ \text{s.t.} \quad & g_1(x) = x_1^2 + x_2^2 - 1 \leq 0, \\ & g_2(x) = x_1 + x_2 - 1 \leq 0. \end{aligned}$$

The feasible set is the unit disk intersected with the half-plane below $x_1 + x_2 = 1$. The unconstrained minimiser $x^u = (2, 0)$ is excluded. The closest feasible point is visually expected to be

$$x^* = (1, 0), \quad f(x^*) = \frac{1}{2}.$$



Objective contours

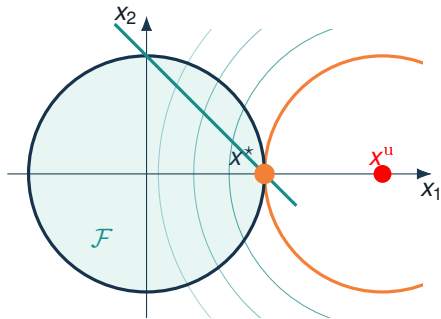
The level sets of f are circles centred at $x^u = (2, 0)$:

$$(x_1 - 2)^2 + x_2^2 = 2c.$$

The first expanding contour to meet the feasible set touches it at $(1, 0)$. Both boundaries pass through that point:

$$g_1(1, 0) = g_2(1, 0) = 0.$$

Hence the active set is $\mathcal{A}(x^*) = \{1, 2\}$.



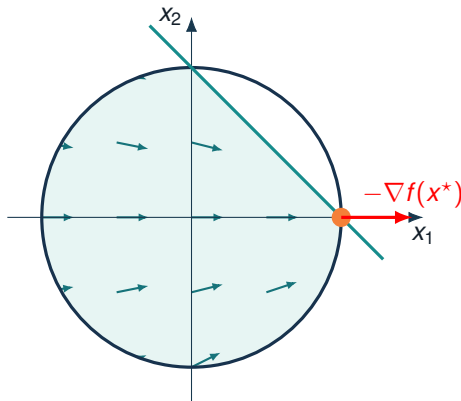
The negative-gradient field is

$$-\nabla f(x) = \begin{bmatrix} 2 - x_1 \\ -x_2 \end{bmatrix}.$$

Every arrow points towards $(2, 0)$. At x^* ,

$$-\nabla f(x^*) = (1, 0)^T,$$

which points outside the disk: the constraint blocks steepest descent.



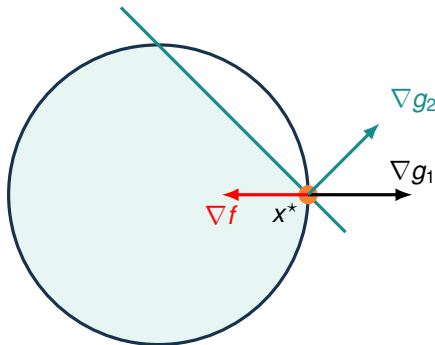
Both constraints active

At $x^* = (1, 0)$,

$$\nabla f(x^*) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \nabla g_1(x^*) = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad \nabla g_2(x^*) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The circle normal is horizontal, directly opposing the objective gradient. The line normal is diagonal.

Activity is a geometric statement: $g_i(x^*) = 0$. Whether an active constraint carries positive multiplier is a separate KKT question.



KKT show a weakly active bound

With multipliers $\mu_1, \mu_2 \geq 0$, the Lagrangian is

$$\mathcal{L}(x, \mu) = f(x) + \mu_1 g_1(x) + \mu_2 g_2(x).$$

Stationarity at $x^* = (1, 0)$ gives

$$\begin{bmatrix} -1 \\ 0 \end{bmatrix} + \mu_1 \begin{bmatrix} 2 \\ 0 \end{bmatrix} + \mu_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 0 \quad \Rightarrow \quad \boxed{\mu_1^* = \frac{1}{2}, \quad \mu_2^* = 0.}$$

Both constraints are active, but only the circle is *strongly active*. The line is *weakly active*: its multiplier vanishes. Therefore strict complementarity fails, although KKT is satisfied.

$\mu_2 \nabla g_2 = 0$

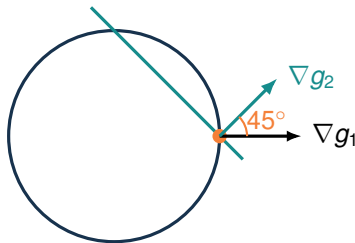
$\nabla f + \mu_1 \nabla g_1 + \mu_2 \nabla g_2 = 0$

LICQ requires the gradients of all active inequalities to be linearly independent. Here

$$J_{\text{act}}(x^*) = \begin{bmatrix} \nabla g_1(x^*)^T \\ \nabla g_2(x^*)^T \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}, \quad \det J_{\text{act}} = 2 \neq 0.$$

Thus $\text{rank } J_{\text{act}} = 2$ and **LICQ holds**.

Geometrically, the circle and the line meet transversally: their normals are not parallel. Consequently the KKT multipliers are unique.



LICQ is a local rank test; it does not require every active multiplier to be positive.

A feasible direction

A feasible direction $d = (d_1, d_2)$ is a vector of the kernel of $J_{\text{act}}(x^*)$ and must satisfy

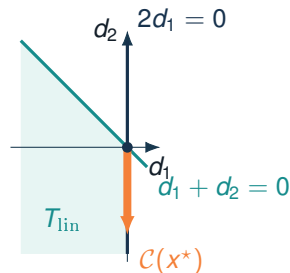
$$2d_1 \leq 0, \quad d_1 + d_2 \leq 0.$$

Criticality also requires $\nabla f(x^*)^T d = -d_1 = 0$. Hence

$$d_1 = 0, \quad d_2 \leq 0,$$

$$\mathcal{C}(x^*) = \{\tau(0, -1)^T : \tau \geq 0\}.$$

The positive circle multiplier enforces equality on the critical cone. The zero-multiplier line keeps a one-sided restriction, so the result is a ray, not a full tangent line.



The Lagrangian Hessian at the KKT point is

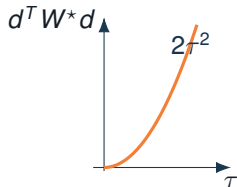
$$\begin{aligned} W^* &= \nabla^2 f(x^*) + \mu_1^* \nabla^2 g_1(x^*) + \mu_2^* \nabla^2 g_2(x^*) \\ &= I_2 + \frac{1}{2}(2I_2) + 0 = 2I_2. \end{aligned}$$

For every nonzero critical direction $d = \tau(0, -1)^T$, $\tau > 0$,

$$d^T W^* d = 2\tau^2 > 0.$$

Second-order sufficient condition

LICQ, KKT and strict positivity of $d^T W^* d$ for every nonzero $d \in \mathcal{C}(x^*)$ imply that x^* is a strict local minimiser.



Convexity $\Rightarrow$ unique global min

The objective is strongly convex, g_1 is convex, and g_2 is affine. Therefore the feasible set is convex and any KKT point is globally optimal; strict convexity makes it unique.

Indeed, convexity of f gives, for every feasible x ,

$$f(x) \geq f(x^*) + \nabla f(x^*)^T (x - x^*) = f(x^*) + 1 - x_1 \geq f(x^*),$$

because $x_1^2 + x_2^2 \leq 1$ implies $x_1 \leq 1$.

Local conclusion

SOSC proves a strict local minimum by testing curvature only on the critical ray.

Global conclusion

Convexity proves that $(1, 0)$ is the unique global minimiser on the whole feasible set.

5. Solvers and algorithms

Problem class: what is the model?

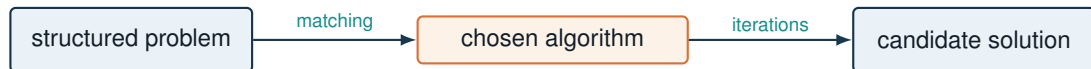
LP, QP, QCQP, SOCP, SDP, LCQP, MINLP,
general NLP

It specifies the algebraic and geometric
structure of the optimisation problem.

Algorithm: how is it solved?

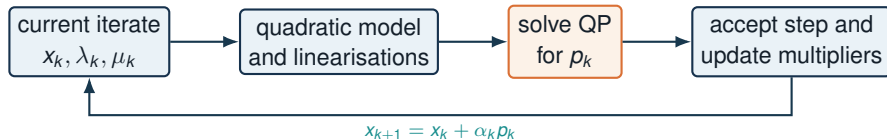
SQP, interior point, active set, trust region,
augmented Lagrangian

It specifies the sequence of computational
steps used to find a solution.



Sequential quadratic programming (SQP) is an algorithm for smooth NLPs. At x_k , it forms a QP in the step p by

- approximating the Lagrangian by a quadratic model;
- linearising the nonlinear constraints;
- solving the QP for a search direction;
- globalising and updating the primal–dual iterate.



At iteration k , solve

$$\begin{aligned} \min_{p \in \mathbb{R}^n} \quad & \nabla f(x_k)^T p + \frac{1}{2} p^T B_k p \\ \text{s.t.} \quad & h(x_k) + J_h(x_k)p = 0, \\ & g(x_k) + J_g(x_k)p \leq 0, \end{aligned}$$

where $B_k \approx \nabla_{xx}^2 \mathcal{L}(x_k, \lambda_k, \mu_k)$.

Exact Hessians yield Newton-like local convergence but may be indefinite away from the solution.

Quasi-Newton updates such as BFGS reduce derivative cost and can preserve positive definiteness.

QP is a problem class; SQP is the outer method that repeatedly solves QPs.

The pure SQP step $x_{k+1} = x_k + p_k$ is reliable only near the solution. Farther away, a globalisation mechanism is required.

Line search

Choose $\alpha_k \in (0, 1]$ so that a merit or filter criterion accepts $x_k + \alpha_k p_k$.

Typical merit functions combine objective reduction and constraint violation.

Globalisation targets convergence from remote initial guesses; the Newton model supplies fast local convergence.

Trust region

Add $\|p\| \leq \Delta_k$ and adapt Δ_k by comparing predicted and actual progress.

Trust regions also regularise indefinite or inaccurate Hessian models.

Primal–dual interior point

Inequalities are approached through positive slacks and perturbed complementarity, with $S = \text{diag}(s)$:

$$g(x) + s = 0, \quad S\mu = \tau \mathbf{1}, \quad s, \mu > 0.$$

Effective for large sparse problems and conic programmes.

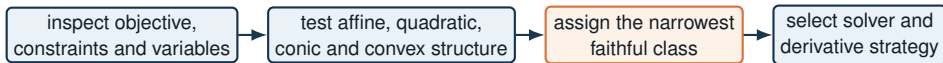
Other families include augmented-Lagrangian, penalty, proximal, first-order and derivative-free methods.

Active set

Predict which inequalities are active, solve the corresponding equality-constrained subproblem, then add or remove constraints.

Often attractive for warm starts and sequences of closely related QPs, as in model predictive control.

Choose the solver wisely



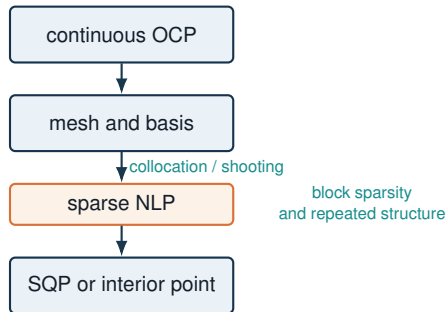
If the model is. . .	Start with. . .
affine	LP solver
convex quadratic with affine constraints	convex QP solver
norm-constrained / robust convex	SOCQP solver
matrix-inequality convex	SDP solver
smooth nonlinear	SQP or nonlinear interior-point solver
mixed continuous–integer	MILP/MIQP/MINLP solver

A continuous-time problem

$$\min_{x(\cdot), u(\cdot)} \Phi(x(T)) + \int_0^T L(x, u) dt$$

subject to $\dot{x} = f(x, u)$ and path/boundary constraints is parameterised on a mesh. The

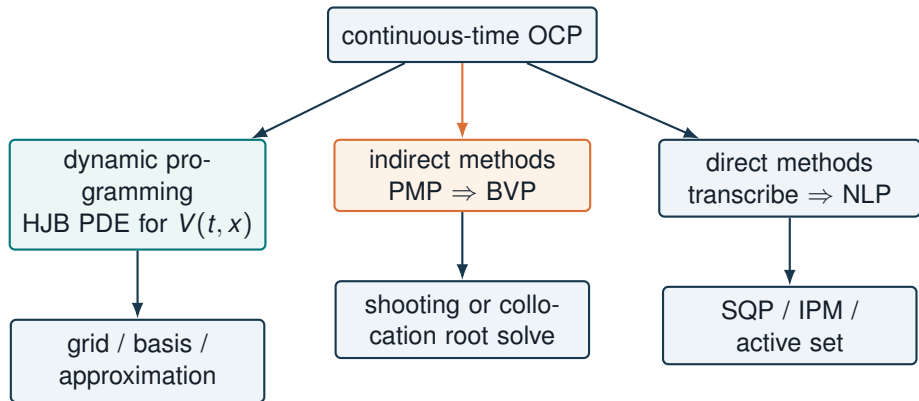
NLP variables contain states, controls, times and possibly parameters.



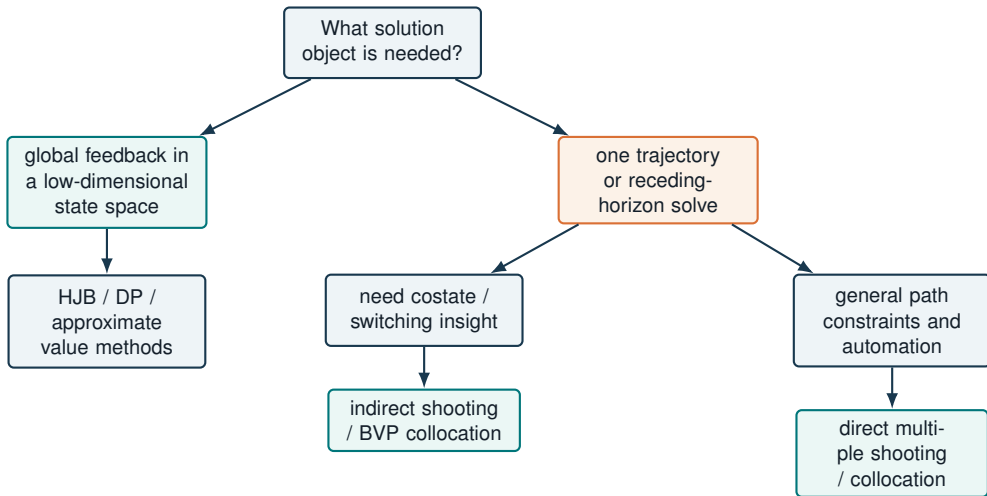
Discretisation determines both the NLP structure and its numerical conditioning.

6. Direct Methods

Three families of numerical OCPs



The families differ mainly in *when* they discretise and *which mathematical object* they approximate.



Optimise then discretise OR vice-versa?

Indirect

$$\begin{aligned} \text{OCP} &\xrightarrow{\text{variation / PMP}} \text{continuous BVP} \\ &\xrightarrow{\text{discretise}} F(w) = 0. \end{aligned}$$

- preserves continuous necessary conditions;
- exposes costates and switching structure.

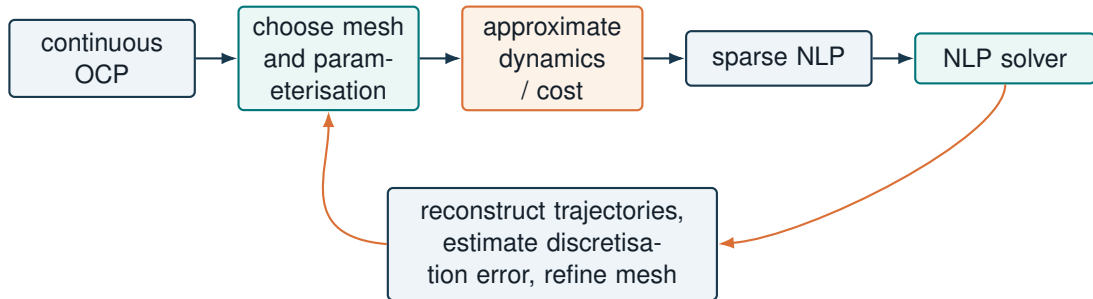
Direct

$$\begin{aligned} \text{OCP} &\xrightarrow{\text{discretise}} \text{finite NLP} \\ &\xrightarrow{\text{optimise}} \text{KKT point.} \end{aligned}$$

- handles inequalities naturally;
- leverages mature NLP solvers.

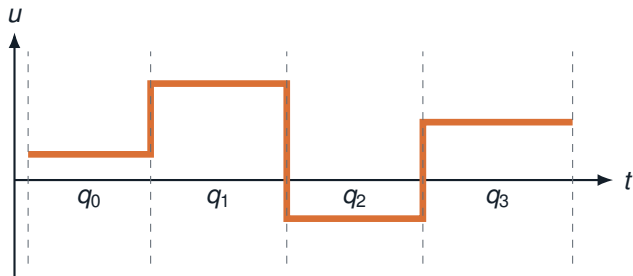
Under consistent discretisation and regularity, the routes are closely related, although their numerical behaviour can differ sharply.

The direct-method pipeline



Partition $[0, T]$ into intervals $[t_k, t_{k+1}]$ and represent

$$u(t) \approx \tilde{u}(t; q).$$



Common choices:

- piecewise constant;
- piecewise linear;
- local polynomials;
- values at collocation nodes;
- global spectral coefficients.

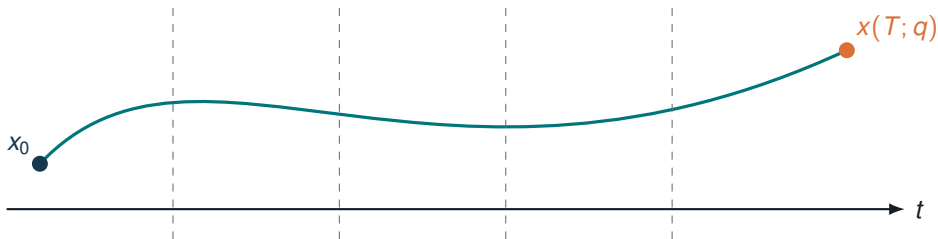
The parameterisation determines both approximation power and the dimension / sparsity of the NLP.

Direct single shooting: concept

Only the control parameters are independent decision variables. The entire state trajectory is generated by one forward simulation:

$$x(t; q) = \varphi(t; x_0, \tilde{u}(\cdot; q)).$$

one coupled simulation over the full horizon



Single shooting is the sequential approach: states are eliminated exactly with respect to the chosen integrator.

Let q parameterise the control and let $\tilde{x}(t; x_0, q)$ be the simulated state. Then

$$\begin{aligned} \min_q \quad & \phi(\tilde{x}(T; x_0, q)) + \sum_{k,j} w_{kj} L(t_{kj}, \tilde{x}_{kj}, \tilde{u}_{kj}) \\ \text{s.t.} \quad & h(t_{kj}, \tilde{x}_{kj}, \tilde{u}_{kj}) \leq 0, \\ & \psi(\tilde{x}(T; x_0, q)) = 0. \end{aligned}$$

- few optimisation variables;
- dynamics are satisfied at every NLP iterate by construction;
- objective and constraints become dense functions of all earlier controls;
- terminal and path constraints require accurate sensitivities.

Strengths

- small number of decision variables;
- always dynamically consistent iterates;
- easy use of sophisticated adaptive simulators;
- excellent for stable systems and good initial guesses.

Weaknesses

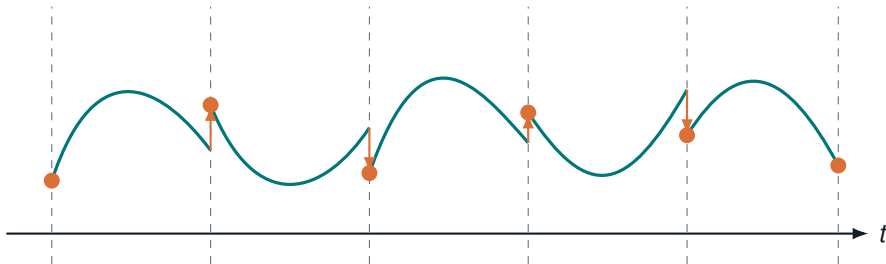
- dense long-horizon sensitivities;
- fragile for unstable dynamics;
- state constraints are hard to satisfy from poor guesses;
- parallelism and sparsity are limited.

Eliminating states reduces dimension but can destroy favourable sparsity and conditioning.

Introduce an independent state variable s_k at every shooting node and integrate only locally:

$$x_{k+1}^- = \Phi_k(s_k, q_k), \quad s_{k+1} - x_{k+1}^- = 0.$$

local arcs need not match before the NLP converges



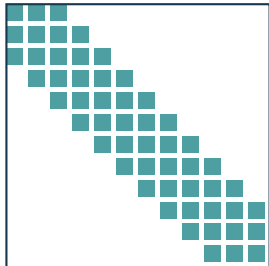
With variables $s_0, \dots, s_N$ and $q_0, \dots, q_{N-1}$:

$$\begin{aligned} \min_{s, q} \quad & \sum_{k=0}^{N-1} \ell_k(s_k, q_k) + \phi(s_N) \\ \text{s.t.} \quad & s_0 = x_0, \\ & s_{k+1} - \Phi_k(s_k, q_k) = 0, \quad k = 0, \dots, N-1, \\ & h_k(s_k, q_k) \leq 0, \quad h_f(s_N) \leq 0. \end{aligned}$$

- more variables and equality constraints than single shooting;
- each interval can use a high-quality ODE / DAE integrator;
- the Jacobian is block-banded in time;
- state guesses can be initialised close to a desired path.

The shooting-defect Jacobian is block-banded:

$$\begin{bmatrix} -I & \Phi_{x,0} & \Phi_{q,0} & & & \\ & -I & \Phi_{x,1} & \Phi_{q,1} & & \\ & & \ddots & \ddots & & \\ & & & -I & \Phi_{x,N-1} & \Phi_{q,N-1} \end{bmatrix}.$$



- each defect touches adjacent states and one local control;
- interval simulations and derivative evaluations can be parallelised;
- sparse factorisation, condensing, or Riccati recursions exploit the temporal chain;
- dense elimination can hide this favourable structure.

Strengths

- robust for unstable dynamics;
- sparse, parallel interval simulations;
- state and path constraints are explicit;
- local arcs can be initialised independently.

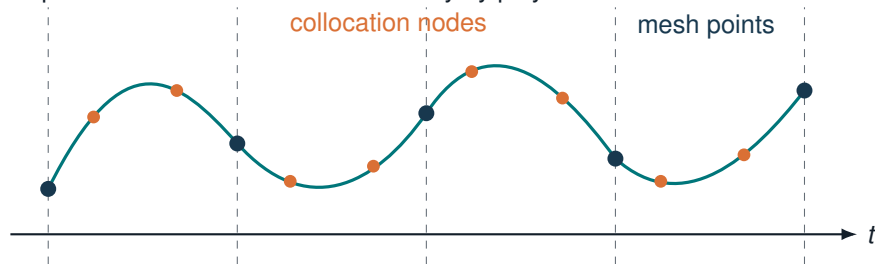
Weaknesses

- larger NLP;
- infeasible state discontinuities during early iterations;
- derivatives of each integrator remain necessary;
- local integration failures can still occur.

Multiple shooting often provides an excellent compromise between robust simulation and sparse simultaneous optimisation.

Direct collocation: optimise poly

Represent both state and control locally by polynomials and enforce the ODE at collocation nodes.



The simulator is no longer a hidden sequential routine: its stage values and residual equations become part of the NLP.

On interval k , map time to $\tau \in [0, 1]$ and write

$$p_k(\tau) = s_k + \sum_{j=1}^M L_j(\tau) S_{k,j},$$

where L_j are basis polynomials and $S_{k,j}$ are internal state variables.

$$\frac{1}{h_k} p'_k(c_j) = f(t_k + c_j h_k, p_k(c_j), u_{k,j}), \quad j = 1, \dots, M.$$

The endpoint map is algebraic: $s_{k+1} = p_k(1)$.

- Lagrange nodal bases make interpolation intuitive;
- orthogonal bases improve high-order conditioning;
- quadrature weights approximate the stage cost integral.

A generic direct-collocation problem has variables

$$(s_0, \dots, s_N), \quad (S_{k,j}), \quad (q_{k,j}),$$

and constraints

$$s_0 = x_0,$$

$$s_{k+1} = F_k(s_k, S_k, q_k),$$

$$0 = G_k(s_k, S_k, q_k),$$

$$H_k(s_k, S_k, q_k) \leq 0.$$

More variables

Internal stage states are explicit NLP variables.

More sparsity

Each constraint touches only one interval and neighbouring endpoints.

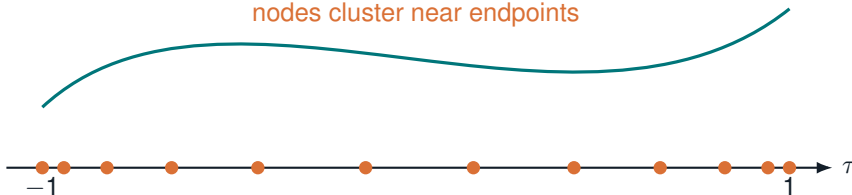
With sparse linear algebra is used, this formulation is very efficient despite its size.

Use one global or a few phase-wise high-degree interpolants at special nodes such as Legendre–Gauss–Lobatto or Radau points:

$$X(\tau) = \sum_{j=0}^M X_j \ell_j(\tau), \quad \sum_{j=0}^M D_{ij} X_j = \frac{T}{2} f(X_i, U_i),$$

where D is a spectral differentiation matrix.

nodes cluster near endpoints



Spectral convergence – and its failure modes

For analytic smooth trajectories, polynomial approximation errors decrease roughly as

$$\|x - x_M\| \lesssim C\rho^{-M}, \quad \rho > 1,$$

which appears exponential in the degree M .

Smooth solution

A modest number of global nodes can achieve very high accuracy.

Nonsmooth solution

Switches, corners, and active-set changes cause slow algebraic convergence and oscillations; split the horizon into phases or use refinement.

Global polynomial coupling also makes the NLP denser than low-order local collocation.

Comparing direct methods

	Single shooting	Multiple shooting	Collocation / spectral
State variables	eliminated	mesh states	mesh and stage states
Dynamics	forward integration	local integrations plus continuity	algebraic defects
Sparsity	reduced but dense	block sparse	very sparse local; denser global
Unstable systems	difficult	strong	strong
DAEs / stiffness	integrator dependent	integrator dependent	natural with implicit collocation
Warm start	controls only	states and controls	full trajectory
Typical use	small stable problems	general nonlinear MPC	large sparse trajectory optimisation

Model stability, path constraints, smoothness, and available sparse linear algebra determine the best transcription.

6. Direct Methods

6.1 Special Cases

Special Cases: Linear systems

For the time-invariant system $\dot{x} = Ax$,

$$x(t) = e^{A(t-t_0)}x(t_0), \quad e^{Ah} = \sum_{j=0}^{\infty} \frac{(Ah)^j}{j!}.$$

Why it matters

e^{Ah} is the exact discrete-time state transition matrix for step h .

How it is computed

Practical algorithms use scaling-and-squaring, Padé approximants, Schur decompositions, or Krylov actions; they do not sum the series naively.

If $A = V\Lambda V^{-1}$ is diagonalisable, $e^{Ah} = Ve^{\Lambda h}V^{-1}$, but this formula can be numerically unsafe when V is ill-conditioned.

For piecewise constant controls u

$$\dot{x} = Ax + Bu, \quad u(t) = u_k \quad t \in [t_k, t_{k+1}),$$

$$x_{k+1} = A_d x_k + B_d u_k, \quad A_d = e^{Ah}, \quad B_d = \int_0^h e^{A\tau} B d\tau.$$

Compute both matrices with one block exponential:

$$\exp\left(\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} h\right) = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix}.$$

Numerical practice

Use a tested `expm`-type routine or a matrix-exponential action. Avoid forming $A^{-1}(A_d - I)B$ when A may be singular or poorly conditioned.

Consider the continuous-time linear system

$$\dot{x}(t) = Ax(t) + Bu(t), \quad x(0) = x_0.$$

The finite-horizon LQR problem is

$$\begin{aligned} \min_{u(\cdot)} \quad & J = \frac{1}{2} x(T)^\top P_f x(T) + \frac{1}{2} \int_0^T (x(t)^\top Q x(t) + u(t)^\top R u(t)) \, dt \\ \text{s.t.} \quad & \dot{x}(t) = Ax(t) + Bu(t), \quad x(0) = x_0. \end{aligned}$$

Assume

$$Q \succeq 0, \quad R \succ 0, \quad P_f \succeq 0.$$

Interpretation: The matrix Q penalises state deviation, R penalises control effort, and P_f penalises the terminal state.

The Hamiltonian is

$$H(x, u, \lambda) = \frac{1}{2}x^\top Qx + \frac{1}{2}u^\top Ru + \lambda^\top (Ax + Bu).$$

Stationarity with respect to u gives

$$H_u = Ru + B^\top \lambda = 0,$$

hence

$$u^*(t) = -R^{-1}B^\top \lambda(t).$$

For LQR, the costate has the form

$$\lambda(t) = P(t)x(t).$$

Therefore

$$u^*(t) = -K(t)x(t), \quad K(t) = R^{-1}B^\top P(t).$$

The matrix $P(t)$ solves the Riccati differential equation

$$-\dot{P} = A^\top P + PA - PBR^{-1}B^\top P + Q, \quad P(T) = P_f.$$

Infinite-horizon case

If $T = \infty$, then $P(t) = P$ is constant and satisfies

$$A^{\top} P + PA - PBR^{-1}B^{\top}P + Q = 0.$$

Consider the nonlinear continuous-time OCP

$$\min_{u(\cdot)} \phi(x(T)) + \int_0^T \ell(x(t), u(t), t) dt,$$

subject to

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(0) = x_0.$$

Main idea

iLQR repeatedly approximates the nonlinear OCP by a local time-varying LQR problem around the current trajectory.



J. B. Rawlings, D. Q. Mayne, and M. M. Diehl.
Model Predictive Control: Theory, Computation, and Design.
2nd ed., Nob Hill Publishing, 2017.



J. Nocedal and S. J. Wright.
Numerical Optimization.
2nd ed., Springer, 2006.



J. T. Betts.
Practical Methods for Optimal Control and Estimation Using Nonlinear Programming.
2nd ed., SIAM, 2010.



L. T. Biegler.
Nonlinear Programming: Concepts, Algorithms, and Applications to Chemical Processes.
MOS-SIAM Series on Optimization, SIAM, 2010.



F. Biral, E. Bertolazzi, and P. Bosetti.
Notes on numerical methods for solving optimal control problems.
IEEJ Journal of Industry Applications, 5(2):154–166, 2016.

September 2026

WHERE: Walk the Future

Thank you for the attention!



marco.frego@unibz.it